Big Data Analytics

-Harsha Gaikwad

What is Big Data

- Big Data meaning **a data that is huge in size**. Bigdata is a term used to describe a collection of data that is huge in size and yet growing exponentially with time.

- Traditional data is measured in Gigabytes(GB) and Terabytes(TB) but Big data is measured in Petabytes and Exabytes(Exabyte=1 Million TB)

- The primary reason to increase the volume of data is dramatic reduction in cost of storage.

Big Data Characteristics

Challenges of Big Data

- Storing Huge Volumes
- Processing Data at huge speed
- Handling variety of forms and functions of data

Application Of Big Data

1. Tracking Customer Spending Habit, Shopping Behavior
2. Recommendation
3. Smart Traffic System
4. Secure Air Traffic System
5. Auto Driving Car
6. Virtual Personal Assistant Tool
7. IoT
8. Education Sector
9. Energy Sector
10. Media and Entertainment Sector

Enabling Technologies for Big Data

Big Data Technologies can be split into two categories

1. Operational Big Data Technologies

2. Analytical Big Data Technologies

1. Operational Big Data Technologies:

- Online ticket bookings, which includes your Rail tickets, Flight tickets, movie tickets etc.
- Online shopping which is your Amazon, Flipkart, Walmart, Snap deal and many more.
- Data from social media sites like Facebook, Instagram, what's app and a lot more.
- The employee details of any Multinational Company.

Enabling Technologies for Big Data

2. Analytical Big Data Technologies

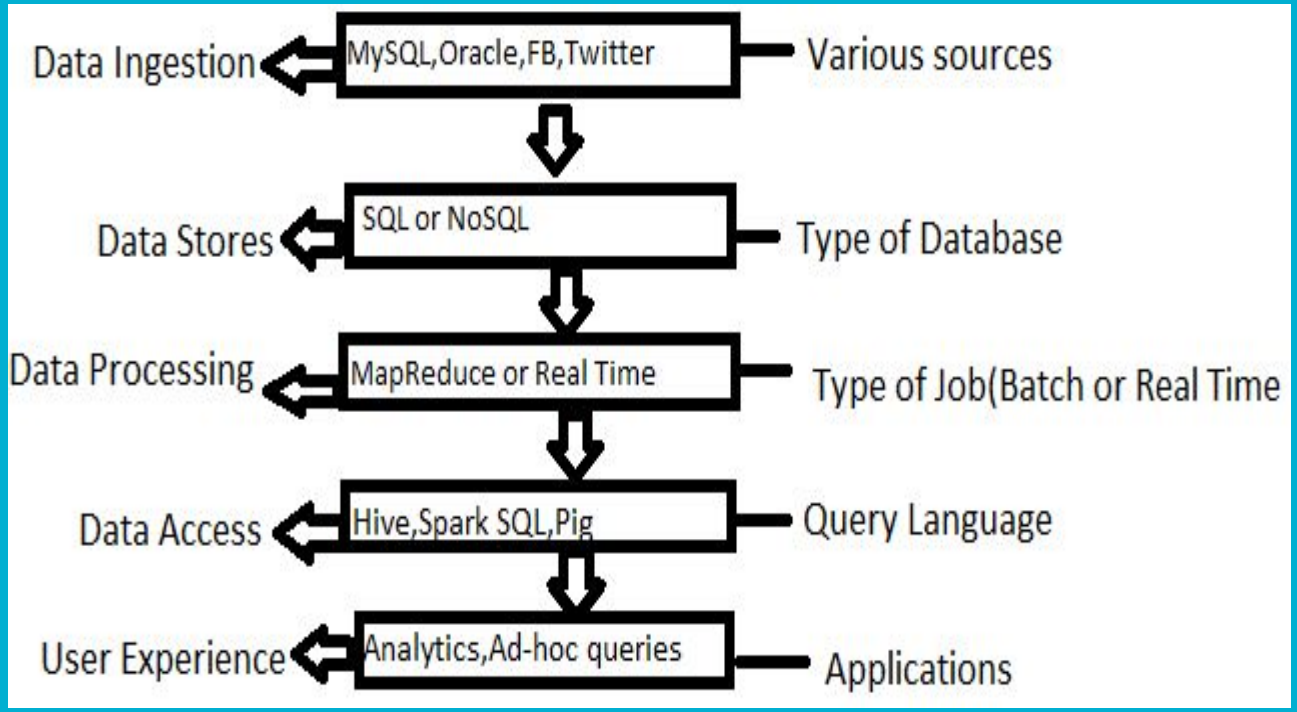
- Stock marketing
- Carrying out the Space missions where every single bit of information is crucial.
- Weather forecast information.
- Medical fields where a particular patients health status can be monitored.

Top Big Data Technologies

1. **Data Storage:**
2. **Data Visualization:**
3. **Data analytics (DA)** : is the process of examining data sets in order to find trends and draw conclusions about the information they contain. Increasingly, data analytics is done with the aid of specialized systems and software.
4. **Data Mining:**Data mining is the process of sorting through large data sets to identify patterns and relationships that can help solve business problems through data analysis.

Top Big Data Technologies

Big Data Stack



HDFC: Hadoop Distributed File System

- HDFS is highly fault-tolerant and is designed to be deployed on low-cost hardware.
- HDFS provides high throughput access to application data and is suitable for applications that have large data sets.

1.Goals And Assumptions

A. Hardware Failure

- Hardware failure is the norm rather than the exception.
- An HDFS instance may consist of hundreds or thousands of server machines, each storing part of the file system's data.
- The fact that there are a huge number of components and that each component has a non-trivial probability of failure means that some component of HDFS is always non-functional.
- **Therefore, detection of faults and quick, automatic recovery from them is a core architectural goal of HDFS.**
- **access.**

HDFC: Hadoop Distributed File System

B. Streaming Data Access

- Applications that run on HDFS need streaming access to their data sets.
- They are not general purpose applications that typically run on general purpose file systems.
- HDFS is designed more for batch processing rather than interactive use by users.
- The emphasis is on high throughput of data access rather than low latency of data access.

C. Large Data Sets

- Applications that run on HDFS have large data sets.
- A typical file in HDFS is gigabytes to terabytes in size. Thus, HDFS is tuned to support large files.
- It should support tens of millions of files in a single instance.

D. Simple Coherency Model

- HDFS applications need a write-once-read-many access model for files.
- A file once created, written, and closed need not be changed.
- This assumption simplifies data coherency issues and enables high throughput data access. A Map/Reduce application or a web crawler application fits perfectly with this model. There is a plan to support appending-writes to files in the future.

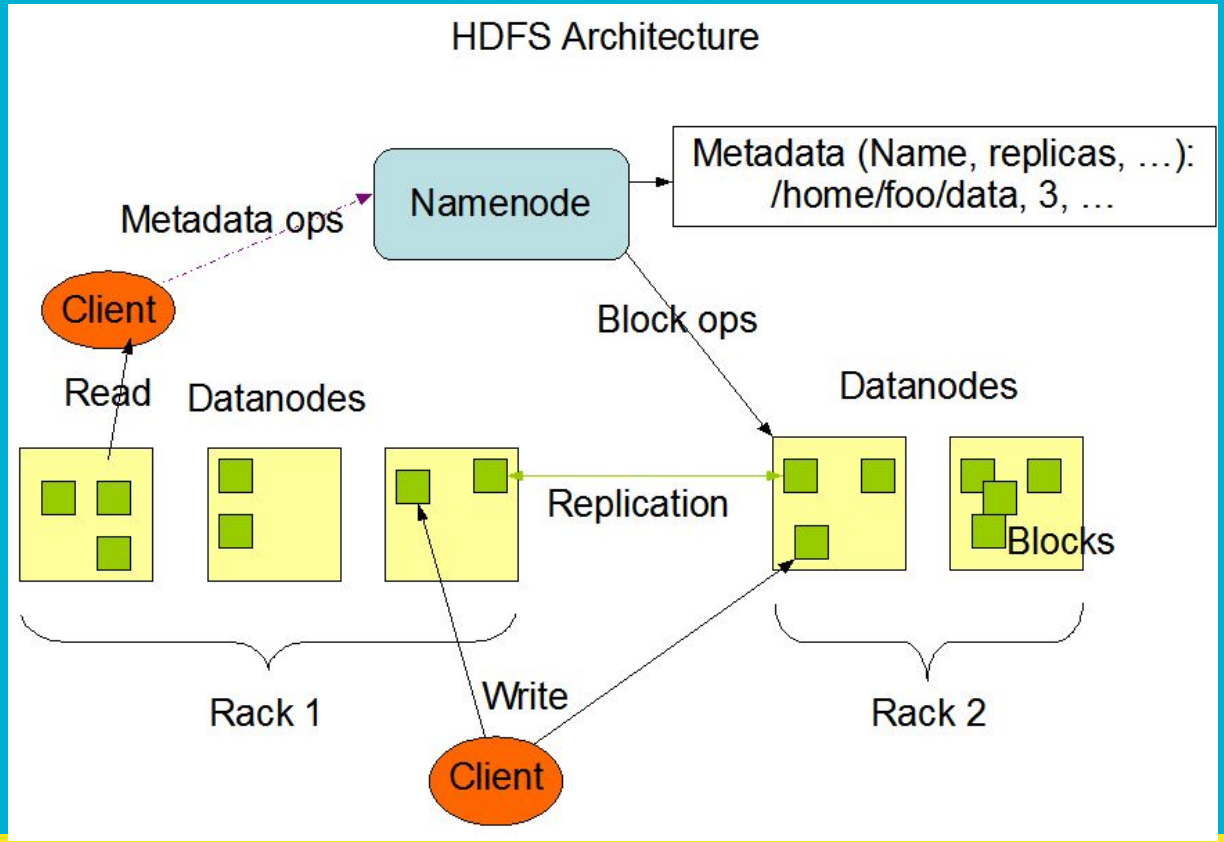
E. **“Moving Computation is Cheaper than Moving Data”**

- A computation requested by an application is much more efficient if it is executed near the data it operates on. This is especially true when the size of the data set is huge.
- This minimizes network congestion and increases the overall throughput of the system. The assumption is that it is often better to migrate the computation closer to where the data is located rather than moving the data to where the application is running. **HDFS provides interfaces for applications to move themselves closer to where the data is located.**

F. Portability Across Heterogeneous Hardware and Software Platforms

HDFS has been designed to be easily portable from one platform to another. This facilitates widespread adoption of HDFS as a platform of choice for a large set of applications.

HDFC Architechture



- HDFS has a master/slave architecture.
- An HDFS cluster consists of a single NameNode,and DAta node

Name Node: a master server that manages the file system namespace and regulates access to files by clients. NameNode executes file system namespace operations like opening, closing, and renaming files and directories. It also determines the mapping of blocks to DataNodes.

Data Nodes: usually one per node in the cluster, which manage storage attached to the nodes that they run on. Internally, a file is split into one or more blocks and these blocks are stored in a set of DataNodes. The The DataNodes are responsible for serving read and write requests from the file system's clients. The DataNodes also perform block creation, deletion, and replication upon instruction from the NameNode.

HDFS exposes a file system namespace and allows user data to be stored in files

- The NameNode and DataNode are pieces of software designed to run on commodity machines.
- These machines typically run a GNU/Linux operating system (OS).
- HDFS is built using the Java language; any machine that supports Java can run the NameNode or the DataNode software.

Data Replication

- HDFS is designed to reliably store very large files across machines in a large cluster.
- It stores each file as a sequence of blocks; all blocks in a file except the last block are the same size.
- The blocks of a file are replicated for fault tolerance.
- The block size and replication factor are configurable per file.
- An application can specify the number of replicas of a file.
- The replication factor can be specified at file creation time and can be changed later.
- Files in HDFS are write-once and have strictly one writer at any time.

The NameNode makes all decisions regarding replication of blocks. It periodically receives a Heartbeat and a Blockreport from each of the DataNodes in the cluster. Receipt of a Heartbeat implies that the DataNode is functioning properly. A Blockreport contains a list of all blocks on a DataNode.

What is MapReduce?

- **MapReduce** is the processing layer of **Hadoop**.
- MapReduce programming model is designed for processing large volumes of data in parallel by dividing the work into a set of independent tasks
- It is the heart of Hadoop.
- Hadoop is so much powerful and efficient due to MapRreduce as here parallel processing is done.
- Map-Reduce divides the work into small parts, each of which can be done in parallel on the cluster of servers.
- A problem is divided into a large number of smaller problems each of which is processed to give individual outputs. These individual outputs are further processed to give final output.

MapReduce Terminologies

- Map-Reduce programs transform lists of input data elements into lists of output data elements.
- A Map-Reduce program will do this twice, using two different list processing idioms-
- Map
- Reduce

What is a MapReduce Job?

- MapReduce Job or a A “full program” is an execution of a **Mapper** and **Reducer** across a data set.
- It is an execution of 2 processing layers i.e mapper and reducer.
- A MapReduce job is a work that the client wants to be performed.
- It consists of the input data, the MapReduce Program, and configuration info. So client needs to submit input data, he needs to write Map Reduce program and set the configuration info (These were provided during **Hadoop setup** in the configuration file and also we specify some configurations in our program itself which will be specific to our **map reduce job)**.

What is Task in Map Reduce?

- A task in MapReduce is an execution of a Mapper or a Reducer on a slice of data.
- It is also called Task-In-Progress (TIP). It means processing of data is in progress either on mapper or reducer.

What is Task Attempt?

- Task Attempt is a particular instance of an attempt to execute a task on a node. There is a possibility that anytime any machine can go down.
- For example, while processing data if any node goes down, framework reschedules the task to some other node. This rescheduling of the task cannot be infinite. There is an upper limit for that as well. The default value of task attempt is 4. If a task (Mapper or reducer) fails 4 times, then the job is considered as a failed job. For high priority job or huge job, the value of this task attempt can also be increased.

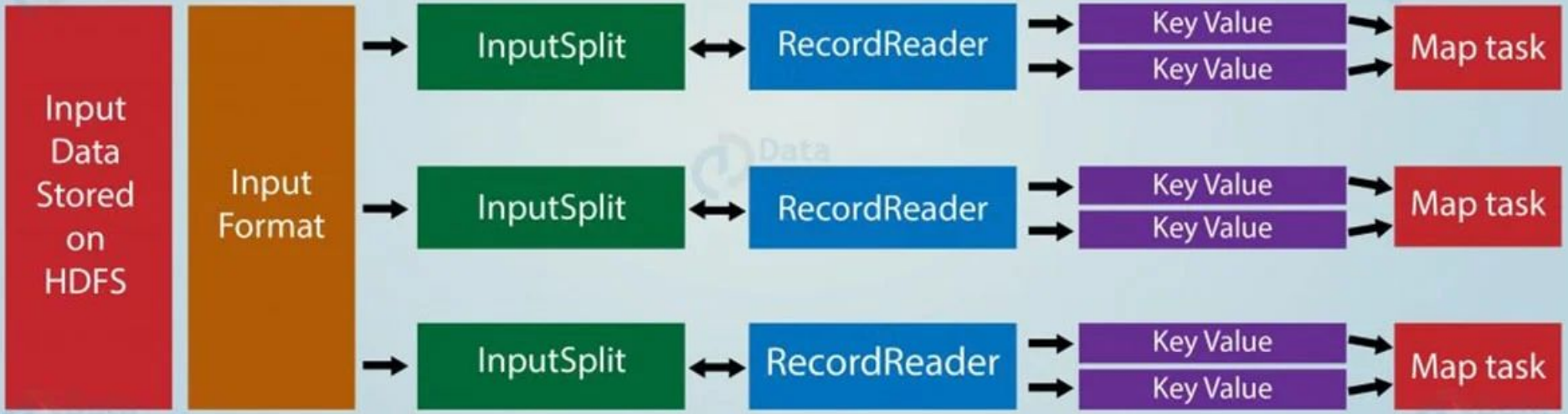
Map Abstraction

- Let us understand the abstract form of **Map** in MapReduce, the first phase of MapReduce paradigm, what is a map/mapper, what is the input to the mapper, how it processes the data, what is output from the mapper?

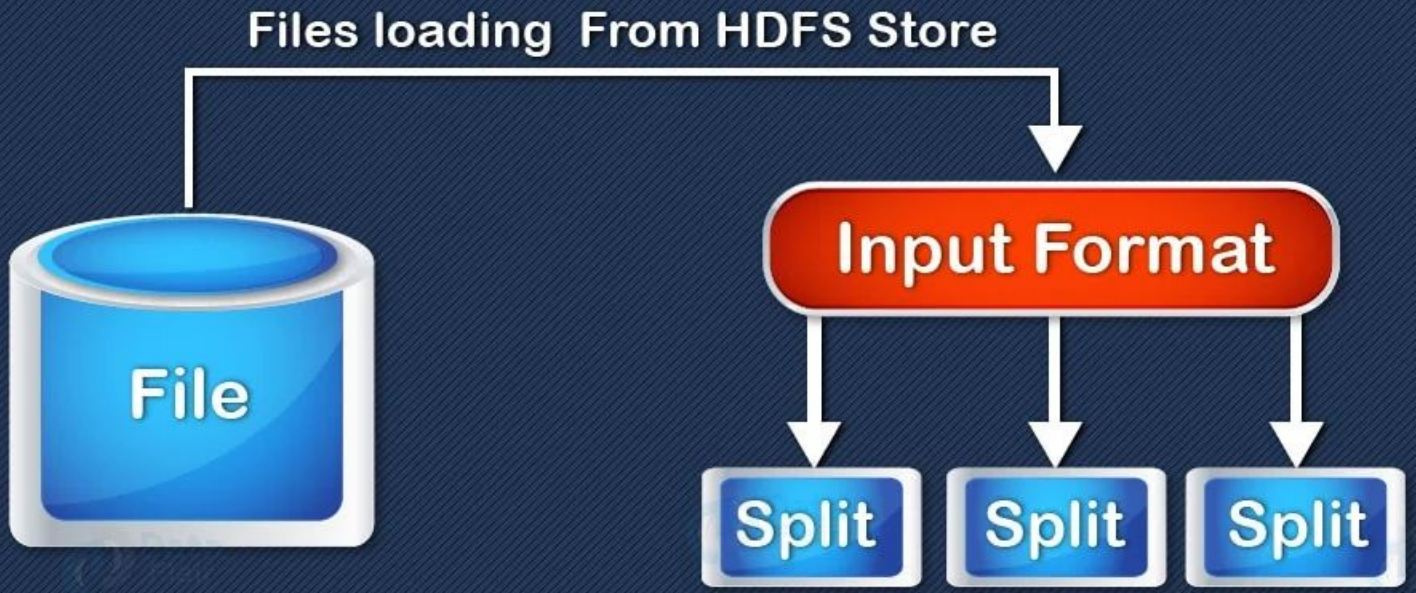
The **map** takes **key/value pair** as input. Whether data is in structured or unstructured format, framework converts the incoming data into key and value.

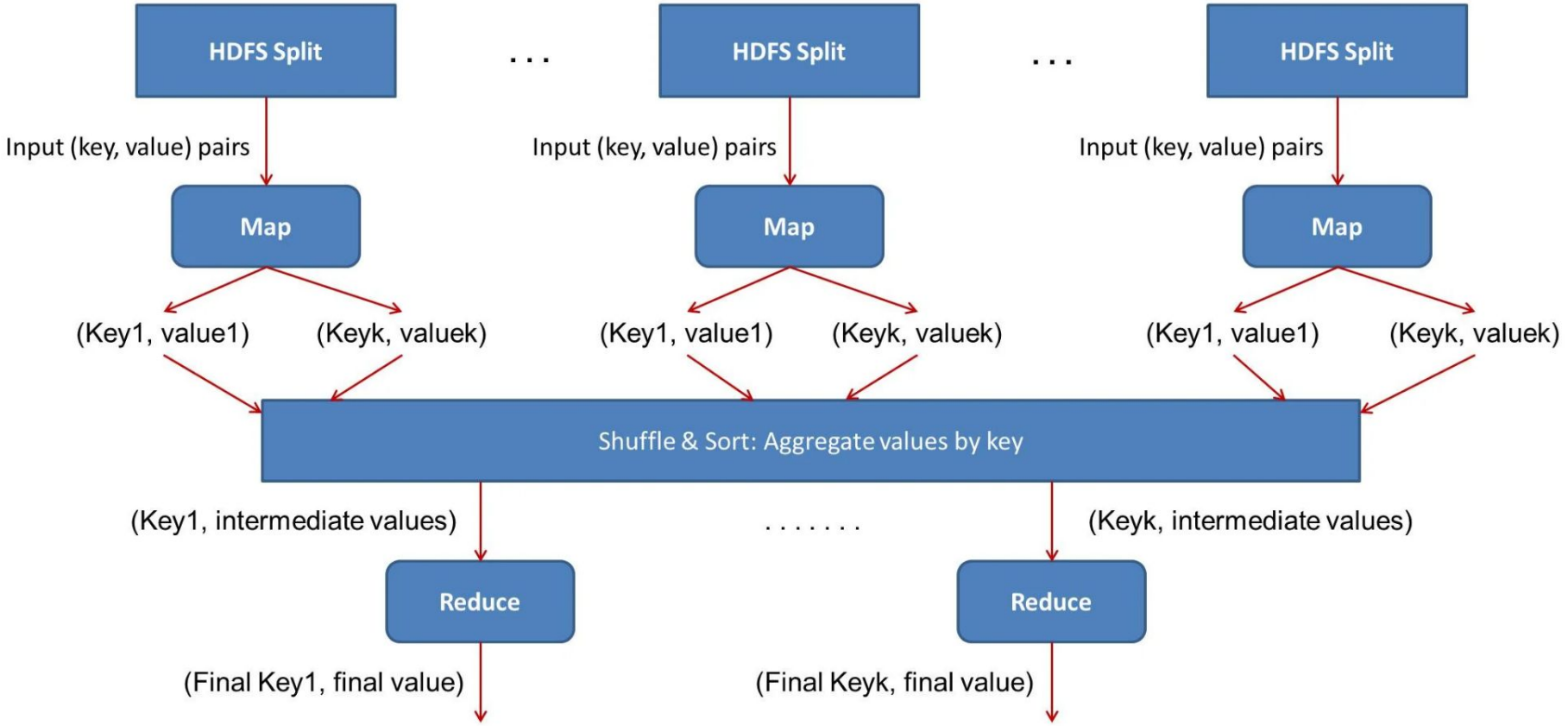
- Key is a reference to the input value.
- Value is the data set on which to operate.

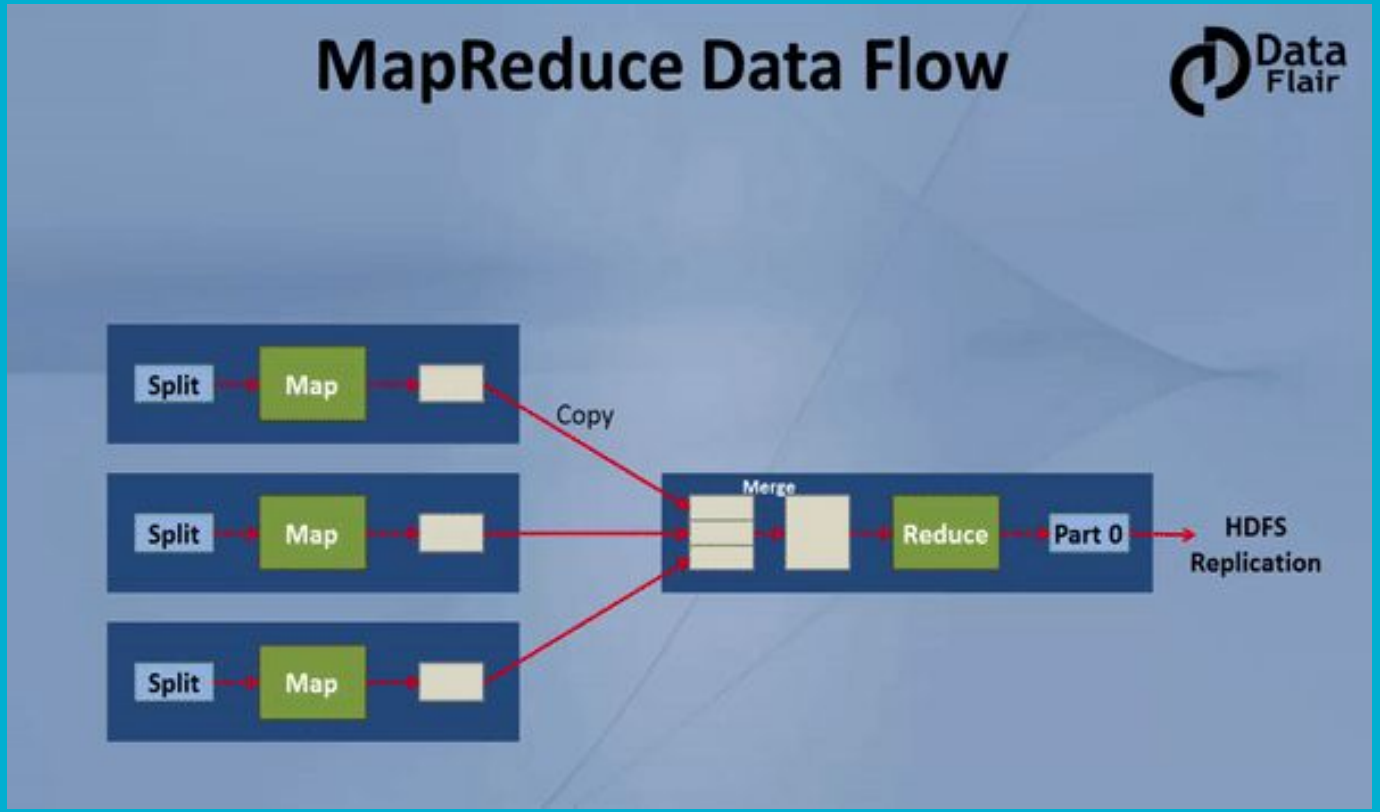
Key Value Pairing in Hadoop MapReduce



InputSplit in Hadoop MapReduce

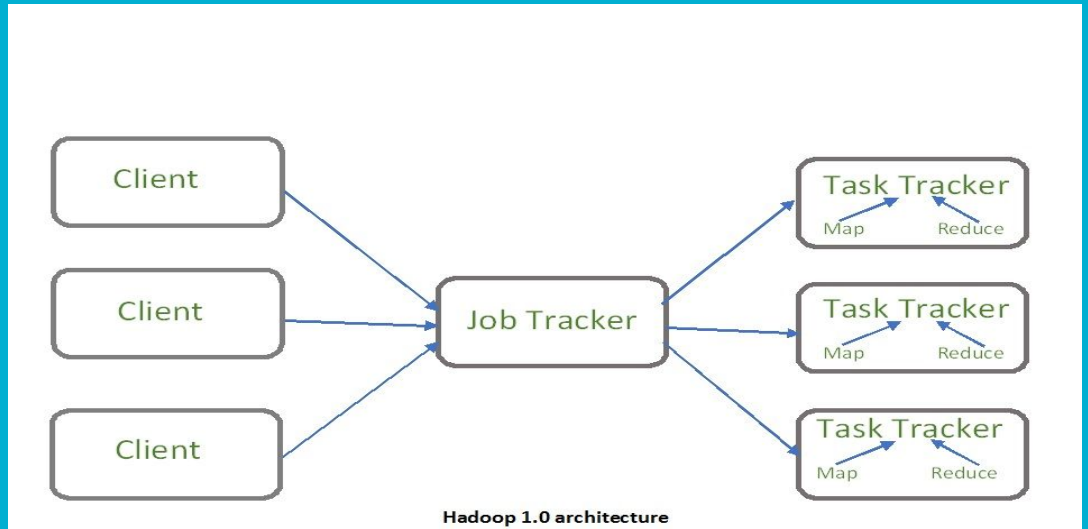



HDFS Split
HDFS Split
HDFS Split
Input (key, value) pairs
Input (key, value) pairs
Input (key, value) pairs
Map
Map
Map
(Key1, value1)
(Keyk, valuek)
(Key1, value1)
(Keyk, valuek)
(Key1, value1)
(Keyk, valuek)
Shuffle & Sort: Aggregate values by key
(Key1, intermediate values)
(Keyk, intermediate values)
Reduce
Reduce
(Final Key1, final value)
(Final Keyk, final value)


MapReduce Data Flow
Data Flair
Split
Map
Split
Map
Split
Map
Copy
Merge
Reduce
Part 0
HDFS Replication

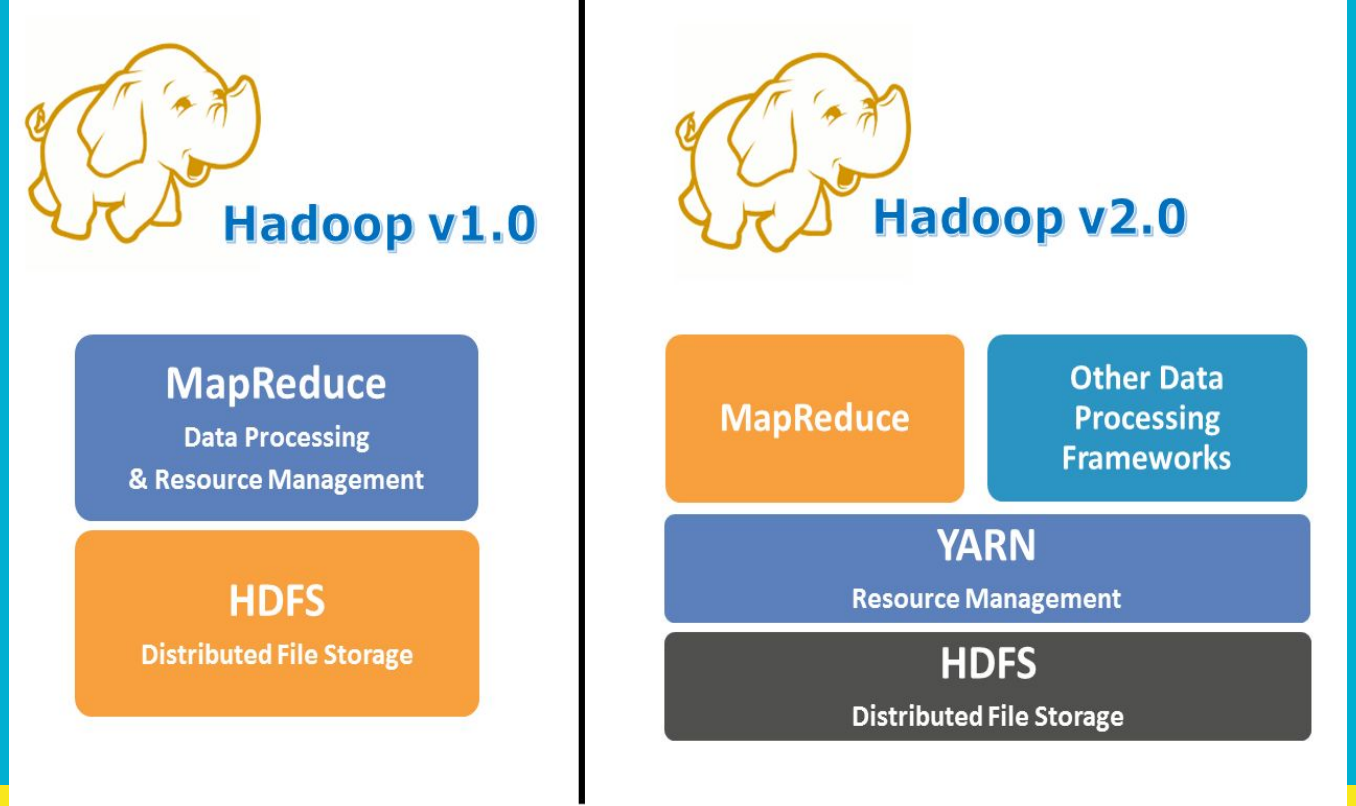
YARN

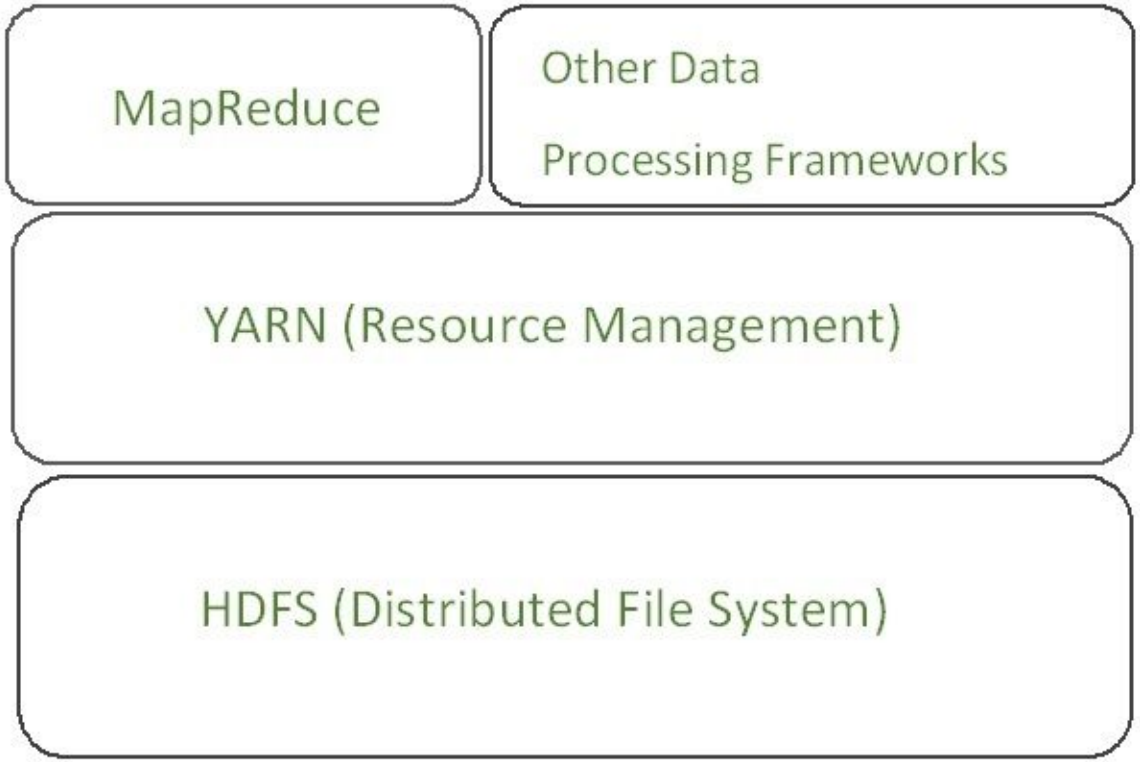
- YARN stands for "***Yet Another Resource Negotiator***".
- It was introduced in Hadoop 2.0 to remove the bottleneck on Job Tracker which was present in Hadoop 1.0.



Hadoop 1.0 architecture

YARN

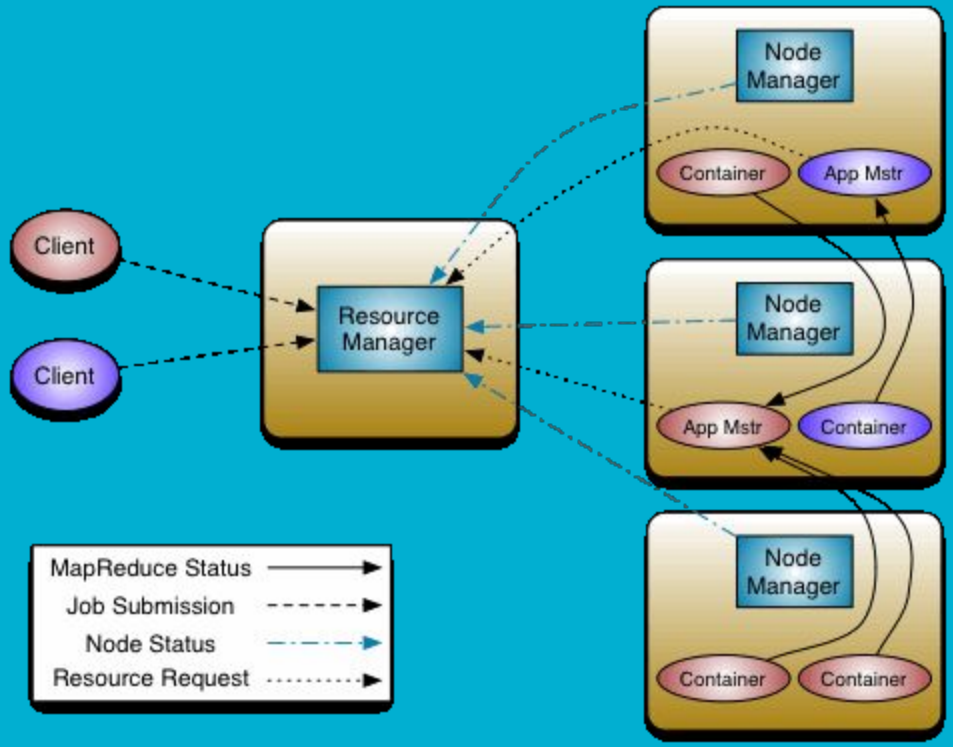

Hadoop v1.0
MapReduce
Data Processing
& Resource Management
HDFS
Distributed File Storage
Hadoop v2.0
MapReduce
Other Data
Processing
Frameworks
YARN
Resource Management
HDFS
Distributed File Storage


MapReduce
Other Data
Processing Frameworks
YARN (Resource Management)
HDFS (Distributed File System)

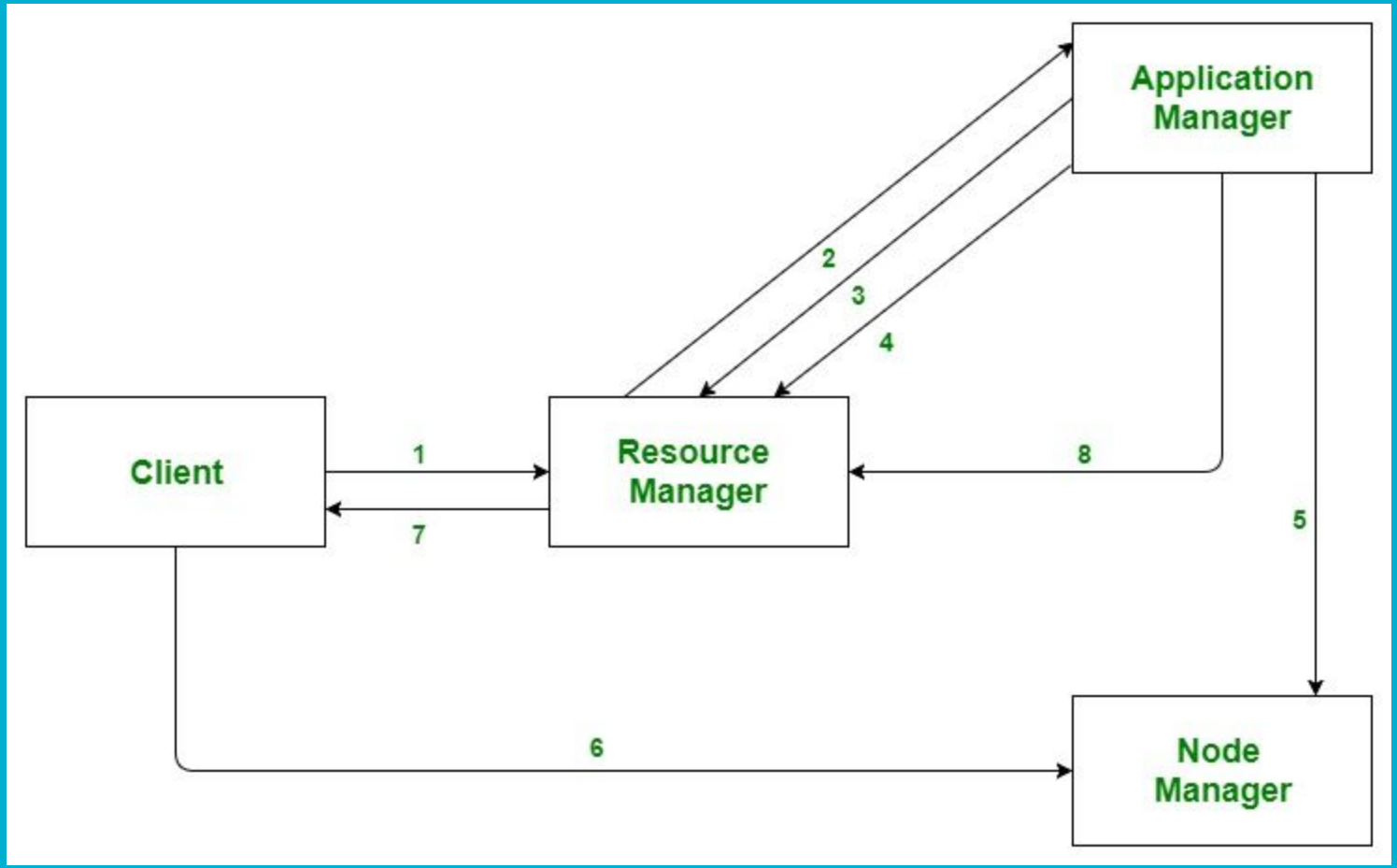
Hadoop 2.0

YARN Features: YARN gained popularity because of the following features-

- **Scalability:** The scheduler in Resource manager of YARN architecture allows Hadoop to extend and manage thousands of nodes and clusters.
- **Compatibility:** YARN supports the existing map-reduce applications without disruptions thus making it compatible with Hadoop 1.0 as well.
- **Cluster Utilization:**Since YARN supports Dynamic utilization of cluster in Hadoop, which enables optimized Cluster Utilization.
- **Multi-tenancy:** It allows multiple engine access thus giving organizations a benefit of multi-tenancy.


Client
Client
Resource Manager
Node Manager
Container
App Mstr
Node Manager
App Mstr
Container
Node Manager
Container
Container
MapReduce Status
Job Submission
Node Status
Resource Request

- The fundamental idea of YARN is to split up the functionalities of resource management and job scheduling/monitoring into separate daemons.
- The idea is to have a global ResourceManager (*RM*) and per-application ApplicationMaster (*AM*). An application is either a single job or a DAG of jobs.
- The ResourceManager and the NodeManager form the data-computation framework.
- The ResourceManager is the ultimate authority that arbitrates resources among all the applications in the system.
- The NodeManager is the per-machine framework agent who is responsible for containers, monitoring their resource usage (cpu, memory, disk, network) and reporting the same to the ResourceManager/Scheduler.
- The per-application ApplicationMaster is, in effect, a framework specific library and is tasked with negotiating resources from the ResourceManager and working with the NodeManager(s) to execute and monitor the tasks.


Application Manager
2
3
4
Client
1
Resource Manager
8
7
5
6
Node Manager

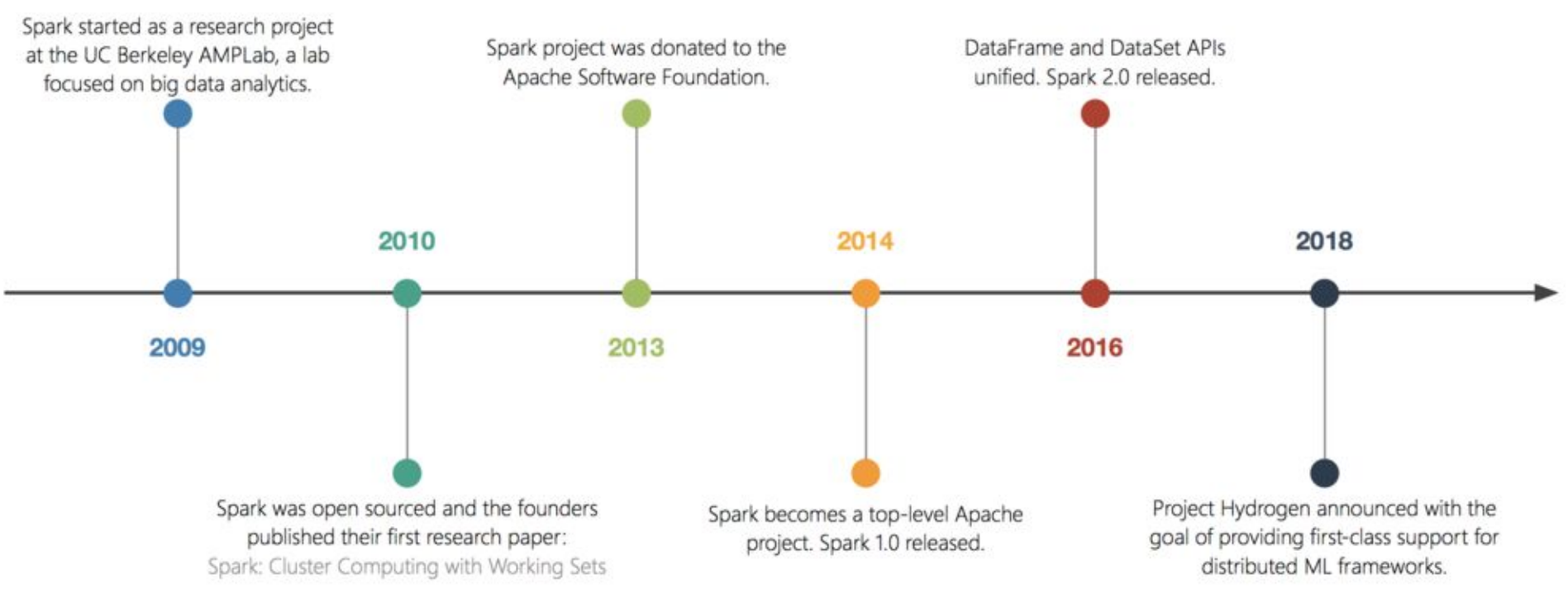
1. Client submits an application
2. The Resource Manager allocates a container to start the Application Manager
3. The Application Manager registers itself with the Resource Manager
4. The Application Manager negotiates containers from the Resource Manager
5. The Application Manager notifies the Node Manager to launch containers
6. Application code is executed in the container
7. Client contacts Resource Manager/Application Manager to monitor application's status
8. Once the processing is complete, the Application Manager un-registers with the Resource Manager

Apache Spark

- Apache Spark is an open-source, distributed processing system used for big data workloads.
- It utilizes in-memory caching, and optimized query execution for fast analytic queries against data of any size.
- It provides development APIs in Java, Scala, Python and R, and supports code reuse across multiple workloads—batch processing, interactive queries, real-time analytics, machine learning, and graph processing.
- You'll find it used by organizations from any industry, including at FINRA, Yelp, Zillow, DataXu, Urban Institute, and CrowdStrike.
- Apache Spark has become one of the most popular big data distributed processing framework with 365,000 meetup members in 2017.

What is the history of Apache Spark?

- Apache Spark started in 2009 as a research project at UC Berkley's AMPLab, a collaboration involving students, researchers, and faculty, focused on data-intensive application domains.
- The goal of Spark was to create a new framework, optimized for fast iterative processing like machine learning, and interactive data analysis, while retaining the scalability, and fault tolerance of Hadoop MapReduce.
- The first paper entitled, "Spark: Cluster Computing with Working Sets" was published in June 2010, and Spark was open sourced under a BSD license. In June, 2013, Spark entered incubation status at the Apache Software Foundation (ASF), and established as an Apache Top-Level Project in February, 2014.
- Spark can run standalone, on Apache Mesos, or most frequently on Apache Hadoop.


Spark started as a research project at the UC Berkeley AMPLab, a lab focused on big data analytics.
2009
2010
Spark was open sourced and the founders published their first research paper:
Spark: Cluster Computing with Working Sets
Spark project was donated to the Apache Software Foundation.
2013
2014
Spark becomes a top-level Apache project. Spark 1.0 released.
DataFrame and DataSet APIs unified. Spark 2.0 released.
2016
2018
Project Hydrogen announced with the goal of providing first-class support for distributed ML frameworks.

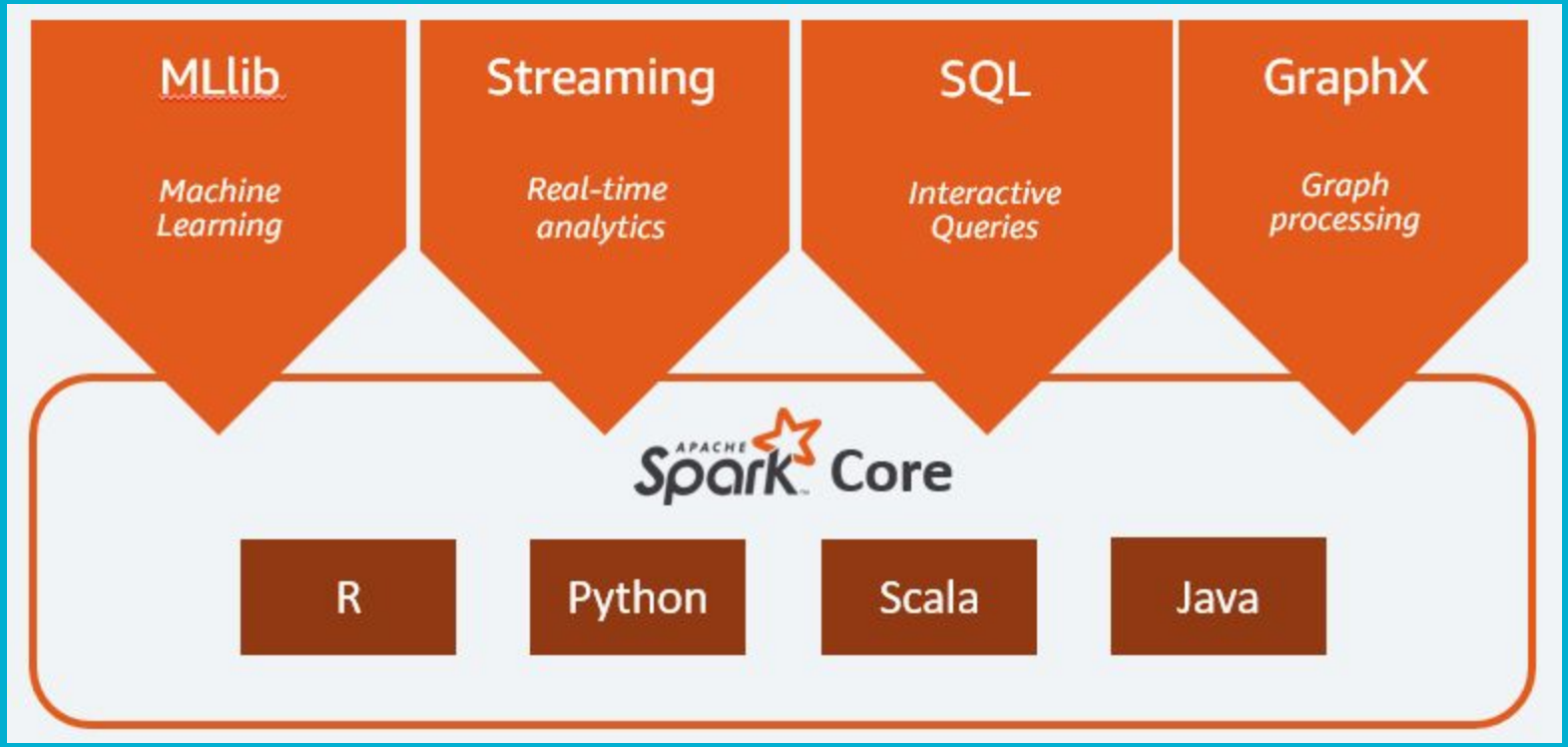
How does Apache Spark work?

- Hadoop MapReduce is a programming model for processing big data sets with a parallel, distributed algorithm. Developers can write massively parallelized operators, without having to worry about work distribution, and fault tolerance.
- However, a challenge to MapReduce is the sequential multi-step process it takes to run a job. With each step, MapReduce reads data from the cluster, performs operations, and writes the results back to HDFS. Because each step requires a disk read, and write, MapReduce jobs are slower due to the latency of disk I/O.

- Spark was created to address the limitations to MapReduce, by doing processing in-memory, reducing the number of steps in a job, and by reusing data across multiple parallel operations.
- With Spark, only one-step is needed where data is read into memory, operations performed, and the results written back—resulting in a much faster execution.
- Spark also reuses data by using an in-memory cache to greatly speed up machine learning algorithms that repeatedly call a function on the same dataset.
- Data re-use is accomplished through the creation of DataFrames, an abstraction over Resilient Distributed Dataset (RDD), which is a collection of objects that is cached in memory, and reused in multiple Spark operations.
- This dramatically lowers the latency making Spark multiple times faster than MapReduce, especially when doing machine learning, and interactive analytics.

Apache Spark vs. Apache Hadoop

- Outside of the differences in the design of Spark and Hadoop MapReduce, many organizations have found these big data frameworks to be complimentary, using them together to solve a broader business challenge.
- Hadoop is an open source framework that has the Hadoop Distributed File System (HDFS) as storage, YARN as a way of managing computing resources used by different applications, and an implementation of the MapReduce programming model as an execution engine. In a typical Hadoop implementation, different execution engines are also deployed such as Spark, Tez, and Presto.
- Spark is an open source framework focused on interactive query, machine learning, and real-time workloads. It does not have its own storage system, but runs analytics on other storage systems like HDFS, or other popular stores like Amazon Redshift, Amazon S3, Couchbase, Cassandra, and others. Spark on Hadoop leverages YARN to share a common cluster and dataset as other Hadoop engines, ensuring consistent levels of service, and response.


MLlib
Machine Learning
Streaming
Real-time analytics
SQL
Interactive Queries
GraphX
Graph processing
APACHE Spark Core
R
Python
Scala
Java

Apache Spark use cases

- **Financial Services**

Spark is used in banking to predict customer churn, and recommend new financial products. In investment banking, Spark is used to analyze stock prices to predict future trends.

- **Healthcare**

Spark is used to build comprehensive patient care, by making data available to front-line health workers for every patient interaction. Spark can also be used to predict/recommend patient treatment.

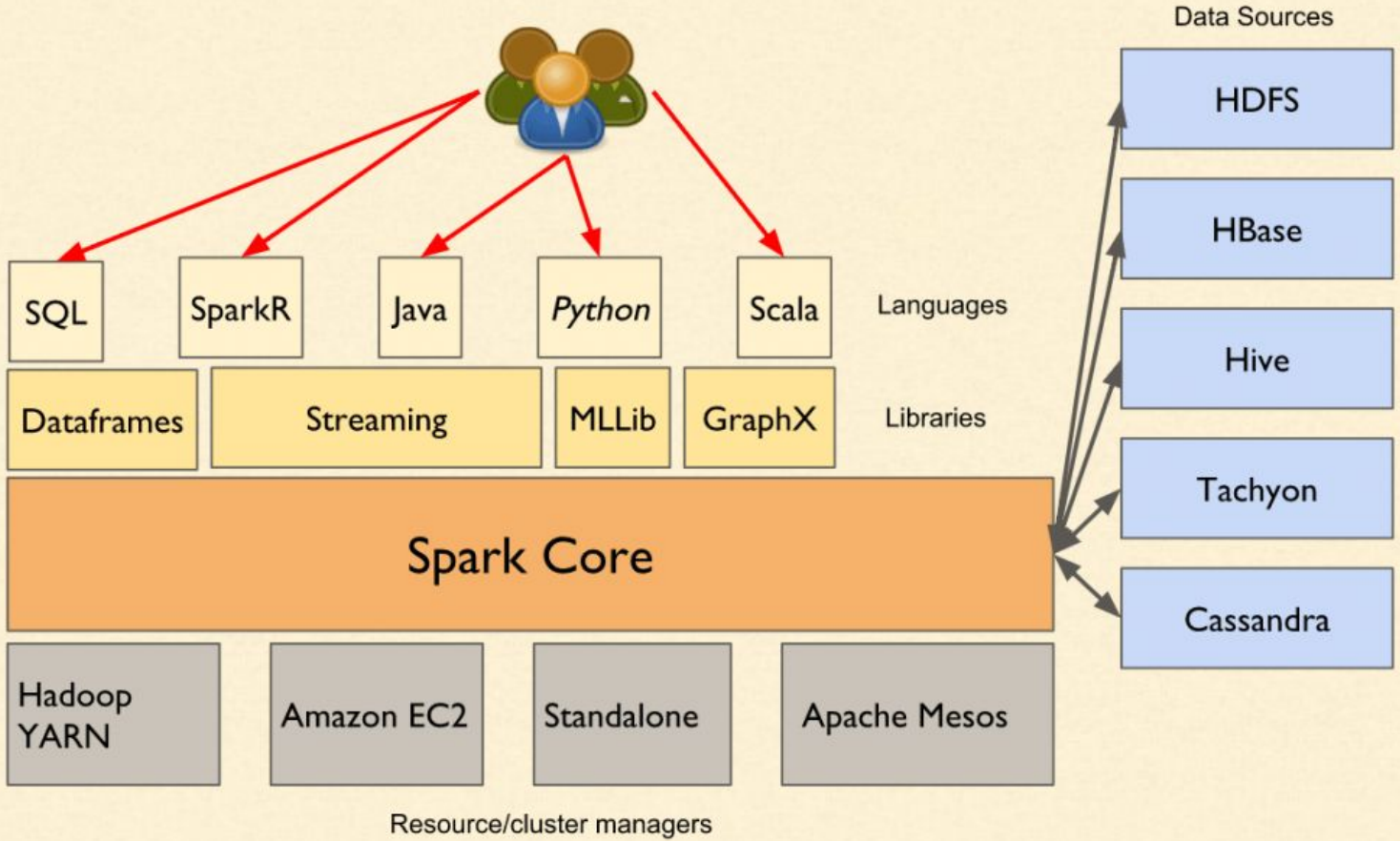
- **Manufacturing**

Spark is used to eliminate downtime of internet-connected equipment, by recommending when to do preventive maintenance.

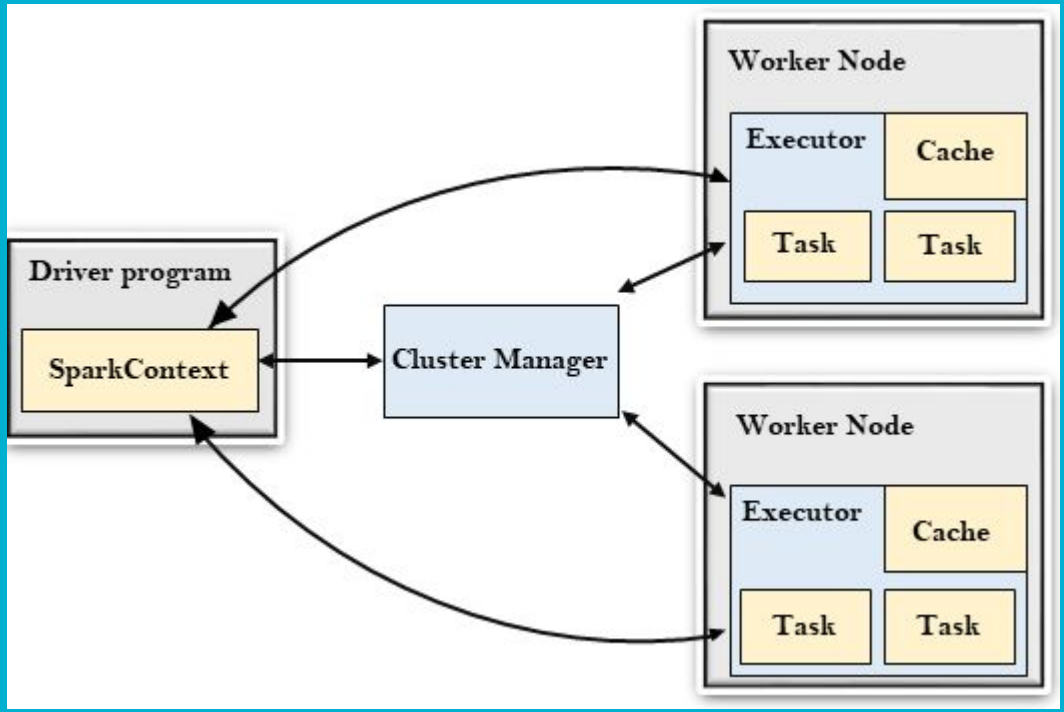
Retail

Spark is used to attract, and keep customers through personalized services and offers.

Spark Architecture



Spark Architecture



The Spark driver

- The driver is the process “in the driver seat” of your Spark Application.
- It is the controller of the execution of a Spark Application and maintains all of the states of the Spark cluster (the state and tasks of the executors).
- It must interface with the cluster manager in order to actually get physical resources and launch executors.
- At the end of the day, this is just a process on a physical machine that is responsible for maintaining the state of the application running on the cluster.

The Spark executors

- Spark executors are the processes that perform the tasks assigned by the Spark driver. Executors have one core responsibility: take the tasks assigned by the driver, run them, and report back their state (success or failure) and results. Each Spark Application has its own separate executor processes.

The cluster manager

- The Spark Driver and Executors do not exist in a void, and this is where the cluster manager comes in.
- The cluster manager is responsible for maintaining a cluster of machines that will run your Spark Application(s).
- Somewhat confusingly, a cluster manager will have its own “driver” (sometimes called master) and “worker” abstractions.

- The core difference is that these are tied to physical machines rather than processes (as they are in Spark). The machine on the left of the illustration is the Cluster Manager Driver Node.
- The circles represent daemon processes running on and managing each of the individual worker nodes. There is no Spark Application running as of yet—these are just the processes from the cluster manager.

- When the time comes to actually run a Spark Application, we request resources from the cluster manager to run it.
- Depending on how our application is configured, this can include a place to run the Spark driver or might be just resources for the executors for our Spark Application.
- Over the course of Spark Application execution, the cluster manager will be responsible for managing the underlying machines that our application is running on.

There are several useful things to note about this architecture:

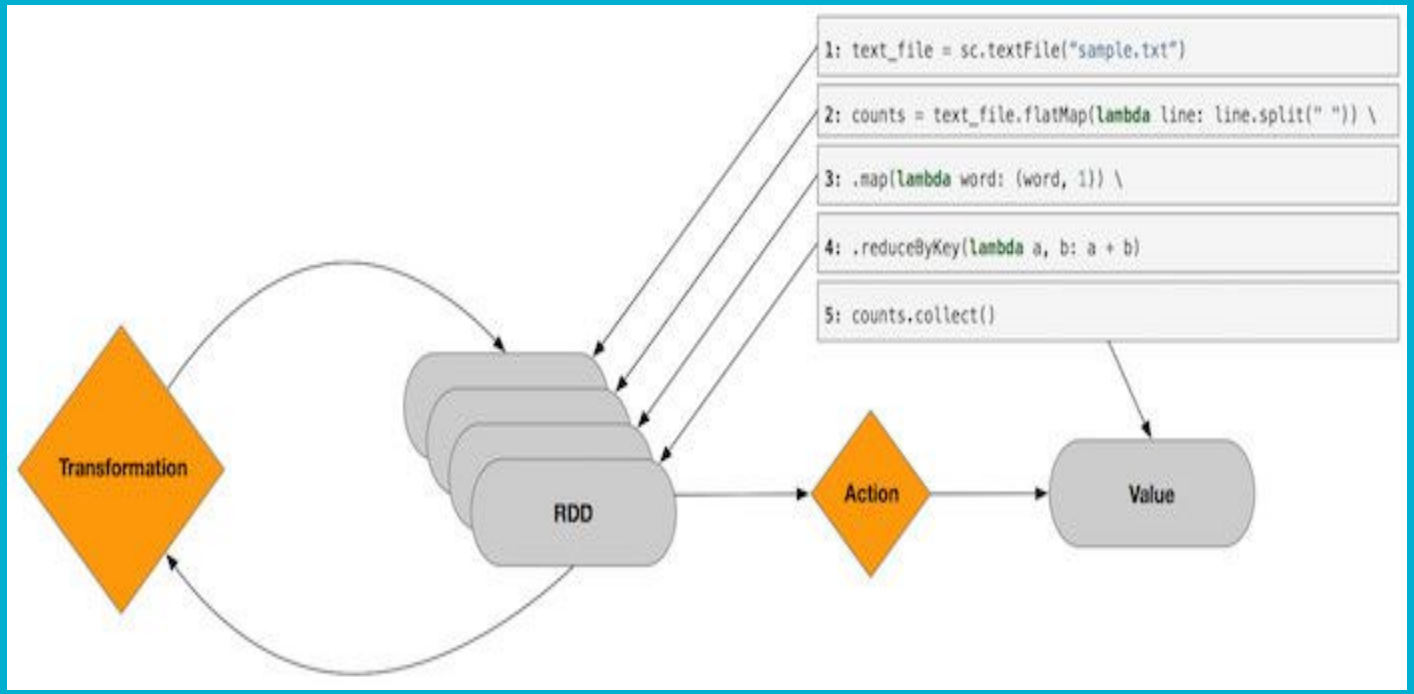
1. Each application gets its own executor processes, which stay up for the duration of the whole application and run tasks in multiple threads.This has the benefit of isolating applications from each other, on both the scheduling side (each driver schedules its own tasks) and executor side (tasks from different applications run in different JVMs).
   However, it also means that data cannot be shared across different Spark applications (instances of SparkContext) without writing it to an external storage system.
2. Spark is agnostic to the underlying cluster manager. As long as it can acquire executor processes, and these communicate with each other, it is relatively easy to run it even on a cluster manager that also supports other applications (e.g. Mesos/YARN).
3. The driver program must listen for and accept incoming connections from its executors throughout its lifetime (e.g., see spark.driver.port in the network config section). As such, the driver program must be network addressable from the worker nodes.
4. Because the driver schedules tasks on the cluster, it should be run close to the worker nodes, preferably on the same local area network. If you'd like to send requests to the cluster remotely, it's better to open an RPC to the driver and have it submit operations from nearby than to run a driver far away from the worker nodes.

Cluster Manager Types

The system currently supports several cluster managers:

- **Standalone** – a simple cluster manager included with Spark that makes it easy to set up a cluster.
- **Apache Mesos** – a general cluster manager that can also run Hadoop MapReduce and service applications.
- **Hadoop YARN** – the resource manager in Hadoop 2.
- **Kubernetes** – an open-source system for automating deployment, scaling, and management of containerized applications.

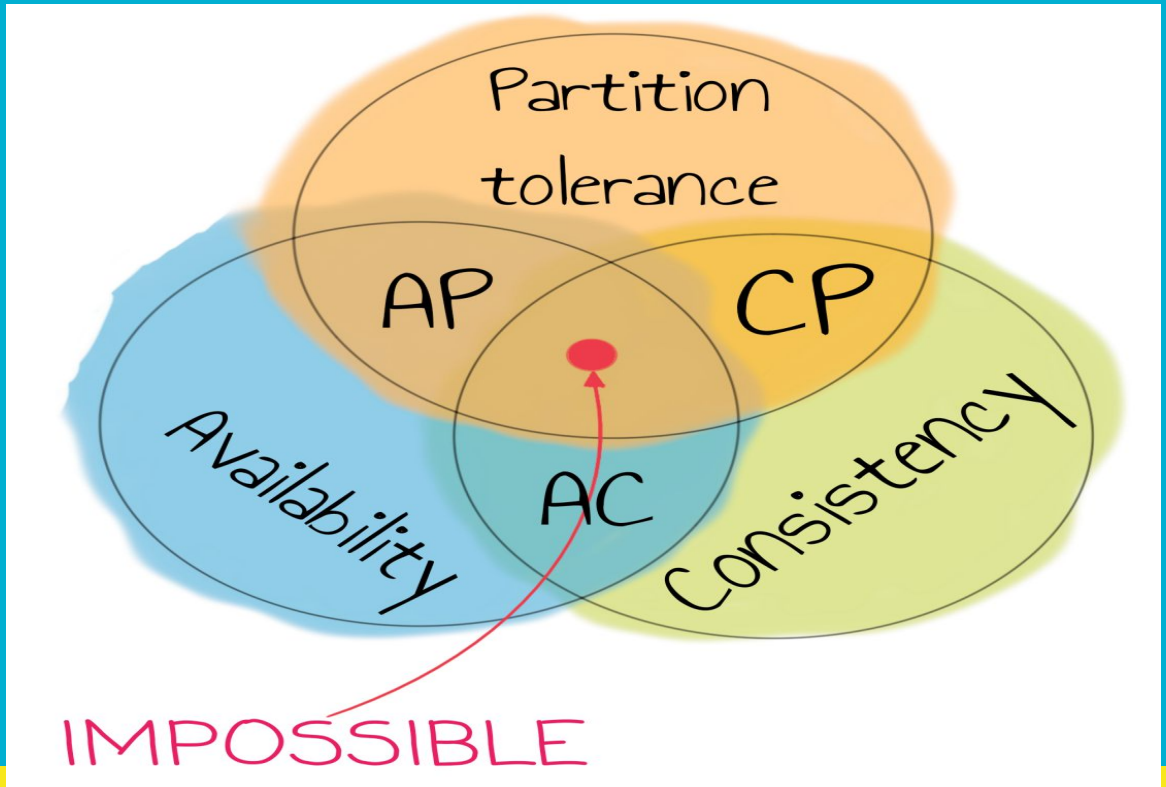
A third-party project (not supported by the Spark project) exists to add support for Nomad as a cluster manager.


1: text_file = sc.textFile("sample.txt")
2: counts = text_file.flatMap(lambda line: line.split(" ")) \
3: .map(lambda word: (word, 1)) \
4: .reduceByKey(lambda a, b: a + b)
5: counts.collect()
Transformation
RDD
Action
Value

CAP Theorem

- A distributed system can deliver only two of three desired characteristics: ***consistency***, ***availability***, and ***partition tolerance*** (the '**C**,' '**A**' and '**P**' in CAP).

- A distributed system is a network that stores data on more than one node (physical or virtual machines) at the same time. Because all cloud applications are distributed systems.


Partition
tolerance
AP
CP
Availability
AC
Consistency
IMPOSSIBLE

Consistency

Consistency means that all clients see the same data at the same time, no matter which node they connect to. For this to happen, whenever data is written to one node, it must be instantly forwarded or replicated to all the other nodes in the system before the write is deemed ‘successful.

Availability

Availability means that that any client making a request for data gets a response, even if one or more nodes are down. Another way to state this—all working nodes in the distributed system return a valid response for any request, without exception.

Partition tolerance

A *partition* is a communications break within a distributed system—a lost or temporarily delayed connection between two nodes. Partition tolerance means that the cluster must continue to work despite any number of communication breakdowns between nodes in the system.

Eventual Consistency

- Eventual Consistency is a guarantee that when an update is made in a distributed database, that update will eventually be reflected in all nodes that store the data, resulting in the same response every time the data is queried.
- Consistency refers to a database query returning the same data each time the same request is made.

ACID Model

1. **Atomicity:** This property states transaction must be treated as an atomic unit, that is, either all of its operations are executed or none.
2. **Consistency:** The database must remain in a consistent state after any transaction also no transaction should have any adverse effect on the data residing in the database and if the database was in a consistent state before the execution of a transaction then it must remain consistent after the execution of the transaction as well.
3. **Isolation:** In a database system where more than one transaction is being executed simultaneously and in parallel, the property of isolation states that each one of the transactions is going to be administered and executed as it is the only transaction in the system also no transaction will affect the existence of any other transactions.
4. **Durability:** The database should be durable enough to hold all its latest updates even if the system fails or restarts so, In a practical way of saying that if a transaction updates a chunk of data in a database and commit is performed then the database will hold the modified data but if the commit is not performed then no data is modified and it can only be done when the

Base Model

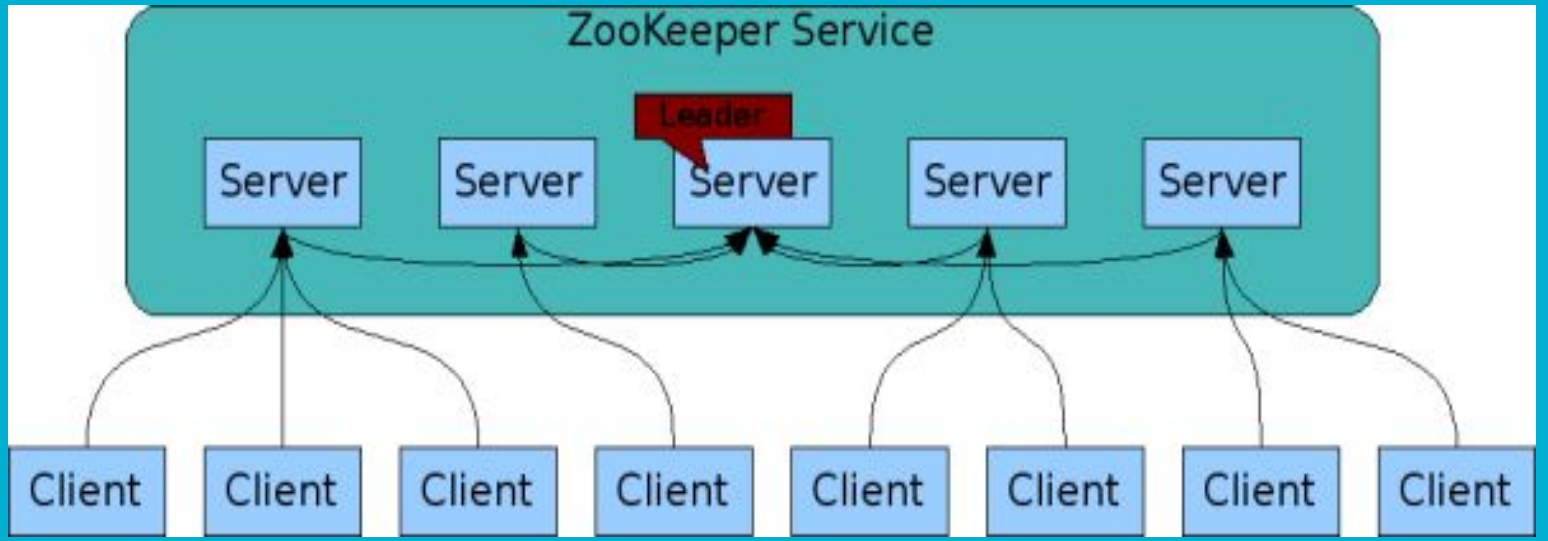
1. **Basically Available:** Instead of making it compulsory for immediate consistency, BASE-modelled NoSQL databases will ensure the availability of data by spreading and replicating it across the nodes of the database cluster.
2. **Soft State:** Due to the lack of immediate consistency, the data values may change over time. The BASE model breaks off with the concept of a database that obligates its own consistency, delegating that responsibility to developers.
3. **Eventually Consistent**: The fact that BASE does not obligates immediate consistency but it does not mean that it never achieves it. However, until it does, the data reads are still possible (even though they might not reflect reality).

ZooKeeper

- Apache ZooKeeper is a service used by a cluster (group of nodes) to coordinate between themselves and maintain shared data with robust synchronization techniques. ZooKeeper is itself a distributed application providing services for writing a distributed application.

The common services provided by ZooKeeper are as follows −

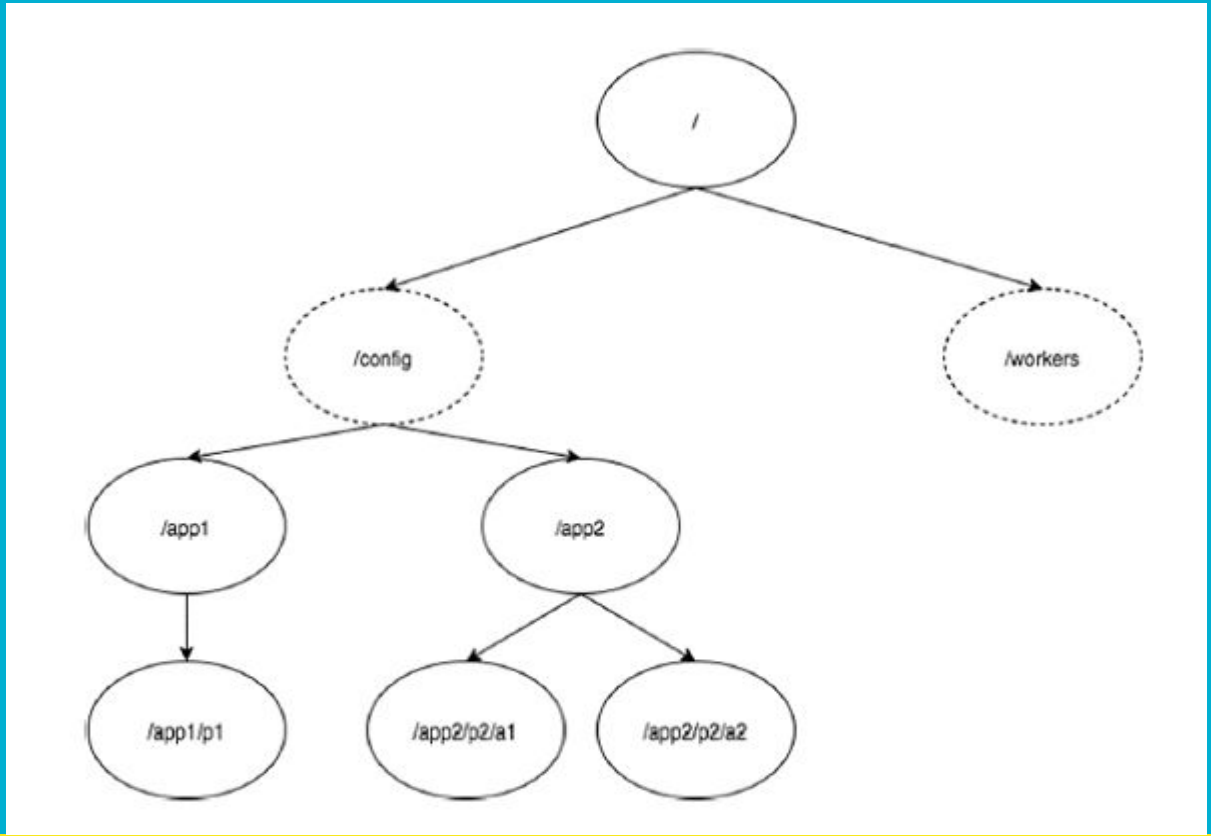
- **Naming service** − Identifying the nodes in a cluster by name. It is similar to DNS, but for nodes.
- **Configuration management** − Latest and up-to-date configuration information of the system for a joining node.
- **Cluster management** − Joining / leaving of a node in a cluster and node status at real time.
- **Leader election** − Electing a node as leader for coordination purpose.
- **Locking and synchronization service** − Locking the data while modifying it. This mechanism helps you in automatic fail recovery while connecting other distributed applications like Apache HBase.
- **Highly reliable data registry** − Availability of data even when one or a few nodes are down.


ZooKeeper Service
Leader
Server
Server
Server
Server
Server
Client
Client
Client
Client
Client
Client
Client
Client

How information is stored in Zookeeper

- Zookeeper stores information in tree like structure
- It is traditional like system like structure. A file is called Znode
- Znode can store data upto 1 MB

ZooKeeper Data Model.



- In the diagram, first you have a root **znode** separated by “/”. Under root, you have two logical namespaces **config** and **workers**.
- The **config** namespace is used for centralized configuration management and the **workers** namespace is used for naming.
- Under **config** namespace, each znode can store upto 1MB of data. This is similar to UNIX file system except that the parent znode can store data as well. The main purpose of this structure is to store synchronized data and describe the metadata of the znode. This structure is called as **ZooKeeper Data Model**.

Every znode in the ZooKeeper data model maintains a **stat** structure. A stat simply provides the **metadata** of a znode. It consists of *Version number, Action control list (ACL), Timestamp, and Data length.*

- **Version number** − Every znode has a version number, which means every time the data associated with the znode changes, its corresponding version number would also increased. The use of version number is important when multiple zookeeper clients are trying to perform operations over the same znode.
- **Action Control List (ACL)** − ACL is basically an authentication mechanism for accessing the znode. It governs all the znode read and write operations.
- **Timestamp** − Timestamp represents time elapsed from znode creation and modification. It is usually represented in milliseconds. ZooKeeper identifies every change to the znodes from “Transaction ID” (zxid). **Zxid** is unique and maintains time for each transaction so that you can easily identify the time elapsed from one request to another request.
- **Data length** − Total amount of the data stored in a znode is the data length. You can store a maximum of 1MB of data.

Types of Znodes

Znodes are categorized as persistence, sequential, and ephemeral.

- **Persistence znode** − Persistence znode is alive even after the client, which created that particular znode, is disconnected. By default, all znodes are persistent unless otherwise specified.
- **Ephemeral znode** − Ephemeral znodes are active until the client is alive. When a client gets disconnected from the ZooKeeper ensemble, then the ephemeral znodes get deleted automatically.
- **Sequential znode** − Sequential znodes can be either persistent or ephemeral. When a new znode is created as a sequential znode, then ZooKeeper sets the path of the znode by attaching a 10 digit sequence number to the original name. For example, if a znode with path **/myapp** is created as a sequential znode, ZooKeeper will change the path to **/myapp0000000001** and set the next sequence number as 0000000002.

Sessions

Sessions are very important for the operation of ZooKeeper. Requests in a session are executed in FIFO order. Once a client connects to a server, the session will be established and a **session id** is assigned to the client.

The client sends **heartbeats** at a particular time interval to keep the session valid. If the ZooKeeper ensemble does not receive heartbeats from a client for more than the period (session timeout) specified at the starting of the service, it decides that the client died.

Watches

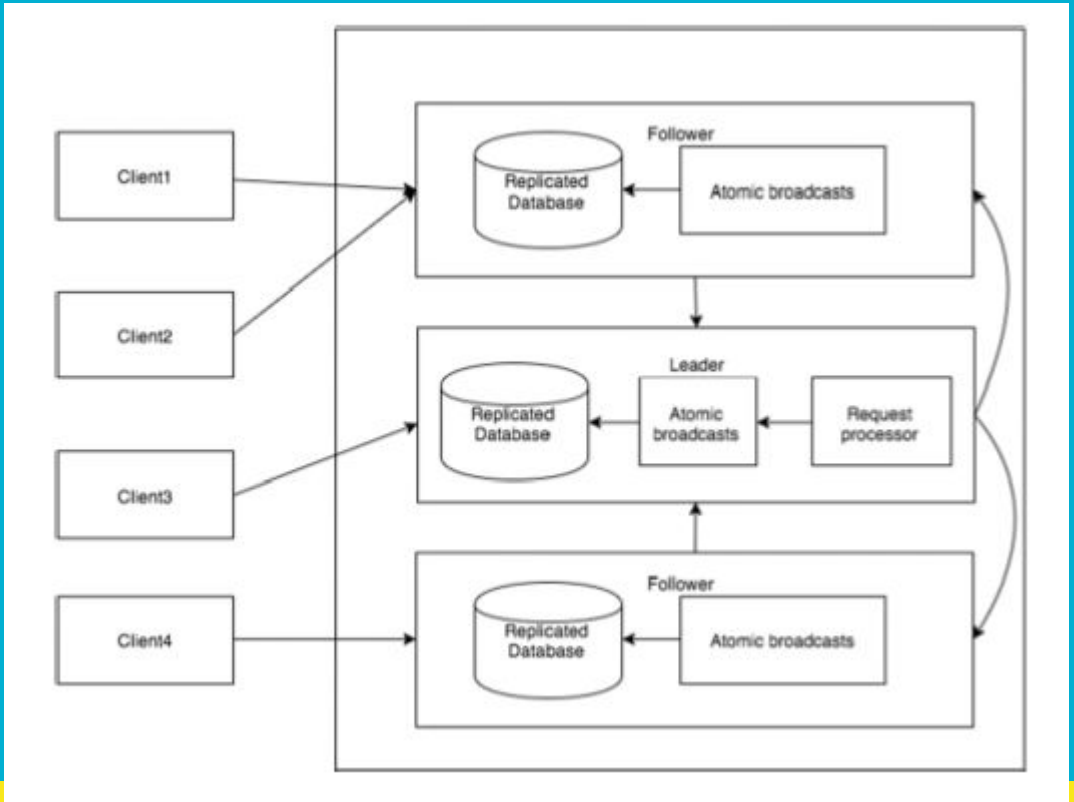
Watches are a simple mechanism for the client to get notifications about the changes in the ZooKeeper ensemble. Clients can set watches while reading a particular znode. Watches send a notification to the registered client for any of the znode (on which client registers) changes.

Nodes in a ZooKeeper Ensemble

Let us analyze the effect of having different number of nodes in the ZooKeeper ensemble.

- If we have **a single node**, then the ZooKeeper ensemble fails when that node fails. It contributes to “Single Point of Failure” and it is not recommended in a production environment.
- If we have **two nodes** and one node fails, we don’t have majority as well, since one out of two is not a majority.
- If we have **three nodes** and one node fails, we have majority and so, it is the minimum requirement. It is mandatory for a ZooKeeper ensemble to have at least three nodes in a live production environment.
- If we have **four nodes** and two nodes fail, it fails again and it is similar to having three nodes. The extra node does not serve any purpose and so, it is better to add nodes in odd numbers, e.g., 3, 5, 7.

Workflow of Zookeeper



Cassandra

- Apache Cassandra is a highly scalable, high-performance distributed database designed to handle large amounts of data across many commodity servers, providing high availability with no single point of failure.
- It is a type of NoSQL database. Let us first understand what a NoSQL database does.

What is Apache Cassandra?

- Apache Cassandra is an open source, distributed and decentralized/distributed storage system (database), for managing very large amounts of structured data spread out across the world. It provides highly available service with no single point of failure.

Features of Cassandra

- Elastic scalability
- Always on architecture
- Fast linear-scale performance
- Flexible data storage
- Easy data distribution
- Transaction support
- Fast writes

History of Cassandra

- Cassandra was developed at Facebook for inbox search.
- It was open-sourced by Facebook in July 2008.
- Cassandra was accepted into Apache Incubator in March 2009.
- It was made an Apache top-level project since February 2010.

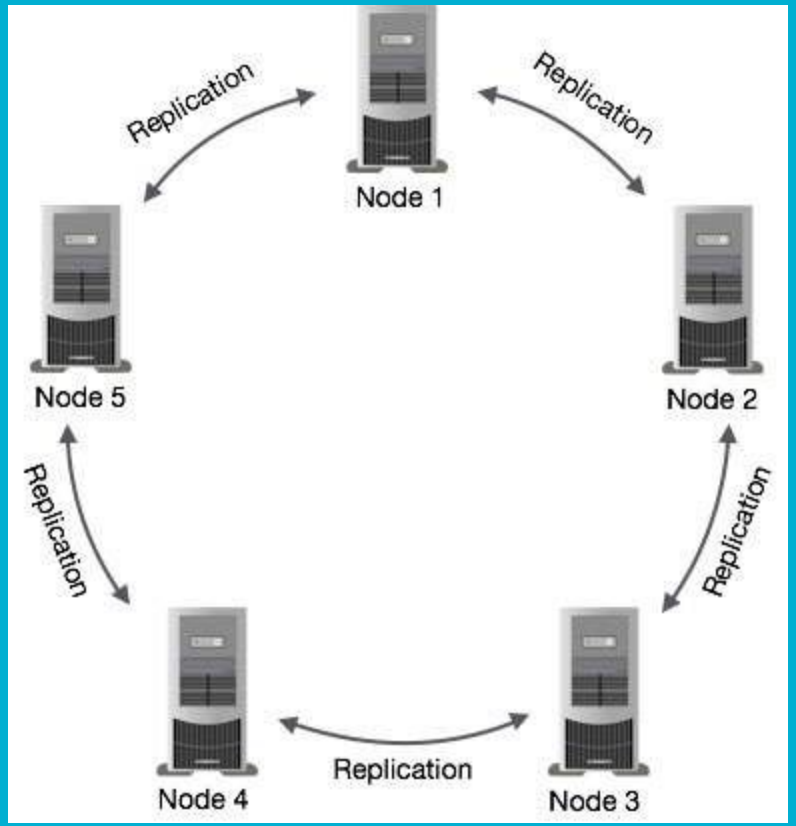
Features of Cassandra

Cassandra has become so popular because of its outstanding technical features. Given below are some of the features of Cassandra:

- **Elastic scalability** − Cassandra is highly scalable; it allows to add more hardware to accommodate more customers and more data as per requirement.
- **Always on architecture** − Cassandra has no single point of failure and it is continuously available for business-critical applications that cannot afford a failure.
- **Fast linear-scale performance** − Cassandra is linearly scalable, i.e., it increases your throughput as you increase the number of nodes in the cluster. Therefore it maintains a quick response time.
- **Flexible data storage** − Cassandra accommodates all possible data formats including: structured, semi-structured, and unstructured. It can dynamically accommodate changes to your data structures according to your need.
- **Easy data distribution** − Cassandra provides the flexibility to distribute data where you need by replicating data across multiple data centers.
- **Transaction support** − Cassandra supports properties like Atomicity, Consistency, Isolation, and Durability (ACID).
- **Fast writes** − Cassandra was designed to run on cheap commodity hardware. It performs blazingly fast writes and can store hundreds of terabytes of data, without sacrificing the read efficiency.

The design goal of Cassandra is to handle big data workloads across multiple nodes without any single point of failure. Cassandra has peer-to-peer distributed system across its nodes, and data is distributed among all the nodes in a cluster.

- All the nodes in a cluster play the same role.
- Each node is independent and at the same time interconnected to other nodes.
- Each node in a cluster can accept read and write requests, regardless of where the data is actually located in the cluster.
- When a node goes down, read/write requests can be served from other nodes in the network.


Node 1
Node 2
Node 3
Node 4
Node 5
Replication
Replication
Replication
Replication
Replication

Components of Cassandra

The key components of Cassandra are as follows −

- **Node** − It is the place where data is stored.
- **Data center** − It is a collection of related nodes.
- **Cluster** − A cluster is a component that contains one or more data centers.
- **Commit log** − The commit log is a crash-recovery mechanism in Cassandra. Every write operation is written to the commit log.
- **Mem-table** − A mem-table is a memory-resident data structure. After commit log, the data will be written to the mem-table. Sometimes, for a single-column family, there will be multiple mem-tables.
- **SSTable** − It is a disk file to which the data is flushed from the mem-table when its contents reach a threshold value.
- **Bloom filter** − These are nothing but quick, nondeterministic, algorithms for testing whether an element is a member of a set. It is a special kind of cache. Bloom filters are accessed after every query.

Write Operations

Every write activity of nodes is captured by the **commit logs** written in the nodes. Later the data will be captured and stored in the **mem-table.** Whenever the mem-table is full, data will be written into the **SStable** data file. All writes are automatically partitioned and replicated throughout the cluster. Cassandra periodically consolidates the SSTables, discarding unnecessary data.

Read Operations

During read operations, Cassandra gets values from the mem-table and checks the bloom filter to find the appropriate SSTable that holds the required data.

What is HBase?

- Apache HBase is an open-source, NoSQL, distributed big data store. It enables random, strictly consistent, real-time access to petabytes of data. HBase is very effective for handling large, sparse datasets.

How does HBase work?

- HBase is a column-oriented, non-relational database. This means that data is stored in individual columns, and indexed by a unique row key.
- This architecture allows for rapid retrieval of individual rows and columns and efficient scans over individual columns within a table.
- Both data and requests are distributed across all servers in an HBase cluster, allowing you to query results on petabytes of data within milliseconds.
- HBase is most effectively used to store non-relational data, accessed via the HBase API. Apache Phoenix is commonly used as a SQL layer on top of HBase allowing you to use familiar SQL syntax to insert, delete, and query data stored in HBase.

Benefits of HBase

Scalable

HBase is designed to handle scaling across thousands of servers and managing access to petabytes of data. With the elasticity of Amazon EC2, and the scalability of Amazon S3, HBase is able to handle online access to massive data sets

Fast

HBase provides low latency random read and write access to petabytes of data by distributing requests from applications across a cluster of hosts. Each host has access to data in HDFS and S3, and serves read and write requests in milliseconds.

Fault-Tolerant

HBase splits data stored in tables across multiple hosts in the cluster and is built to withstand individual host failures. Because data is stored on HDFS or S3, healthy hosts will automatically be chosen to host the data once served by the failed host, and data is brought online automatically.

Data Streaming

- **Data Streaming** is all about communication between a sender and receiver via a single data stream or multiple data streams.
- Such streams are nothing but a sequence of digitally encoded signals ensuring that data traversing from source to destination is continuously analyzed and processed in real-time and at higher data transfer speed.

Top 7 Real-Time Data Streaming Tools

1. Azure Stream Analytics
2. Amazon Kinesis
3. Apache Kafka
4. IBM Stream Analytics
5. Confluent
6. Google Cloud Dataflow
7. Striim

Link : https://www.geeksforgeeks.org/top-7-real-time-data-streaming-tools/

Big Data Pipeline for real time computing

- Big Data Pipelines should **transform, ingest, and analyze data in near real-time so that businesses can quickly find and act on insights**.
- In a single sentence, to build up an efficient big data analytic system for enabling organizations to make decisions on the fly.
- In a real-time big data pipeline, you need to consider factors like real-time fraud analysis, log analysis, predicting errors to measure the correct business decisions.

Why is Real-time Big Data Pipeline So Important Nowadays?

- Helps to make operational decisions.
- The decisions built out of the results will be applied to business processes, different production activities, and transactions in real-time.
- It can be applied to prescriptive or pre-existing models.
- Helps to generate historical and current data concurrently.
- Generates alerts based on predefined parameters.
- Monitors constantly for changing transactional data sets in real-time

What are the Different Features of a Real-time Big Data Pipeline System?

A real-time big data pipeline should have some essential features to respond to business demands, and besides that, it should not cross the cost and usage limit of the organization.

High volume data storage: The system must have a robust big data framework like Apache Hadoop.

Messaging system: It should have publish-subscribe messaging support like Apache Kafka.

Predictive analysis support: The system should support various machine learning algorithms. Hence it must have required library support like Apache Spark MLlib.

The flexible backend to store result data: The processed output must be stored in some database. Hence, a flexible database preferably NoSQL data should be in place.

Reporting and visualization support: The system must have some reporting and visualization tool like Tableau.

Alert support: The system must be able to generate text or email alerts, and related tool support must be in place.

Why are Apache Hadoop, Apache Spark, and Apache Kafka the Choices for Real-time Big Data Pipeline?

There are some key points that we need to measure while selecting a tool or technology for building a big data pipeline which is as follows:

- Components
- Parameters

Components of a big data pipeline are:

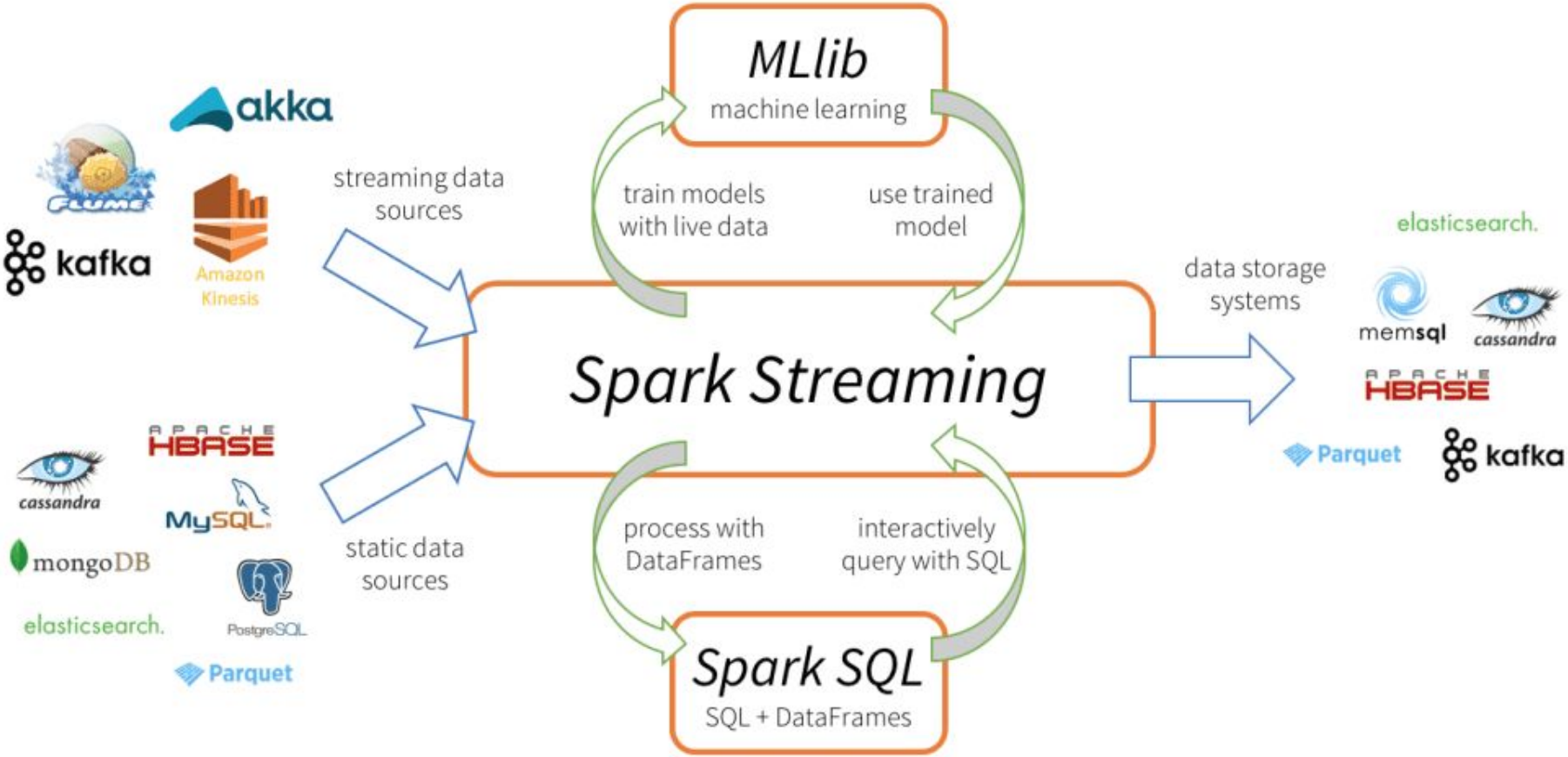
- The messaging system.
- Message distribution support to various nodes for further data processing.
- Data analysis system to derive decisions from data.
- Data storage system to store results and related information.
- Data representation and reporting tools and alerts system.

Important parameters that a big data pipeline system must have –

- Compatible with big data
- Low latency
- Scalability
- A diversity that means it can handle various use cases
- Flexibility
- Economic

What is Spark Streaming?

- Apache Spark Streaming is a scalable fault-tolerant streaming processing system that natively supports both batch and streaming workloads.
- Spark Streaming is an extension of the core Spark API that allows data engineers and data scientists to process real-time data from various sources including (but not limited to) Kafka, Flume, and Amazon Kinesis
- This processed data can be pushed out to file systems, databases, and live dashboards.
- Its key abstraction is a Discretized Stream or, in short, a DStream, which represents a stream of data divided into small batches.
- DStreams are built on RDDs, Spark's core data abstraction. This allows Spark Streaming to seamlessly integrate with any other Spark components like MLlib and Spark SQL. S
- park Streaming is different from other systems that either have a processing engine designed only for streaming, or have similar batch and streaming APIs but compile internally to different engines. Spark's single execution engine and unified programming model for batch and streaming lead to some unique benefits over other traditional streaming systems.


MLlib
machine learning
train models with live data
use trained model
Spark Streaming
process with DataFrames
interactively query with SQL
Spark SQL
SQL + DataFrames
streaming data sources
static data sources
data storage systems
akka
flume
kafka
Amazon Kinesis
cassandra
HBASE
MySQL
mongoDB
elasticsearch.
PostgreSQL
Parquet
memsql
cassandra
HBASE
Parquet
kafka
elasticsearch.

Four Major Aspects of Spark Streaming

- Fast recovery from failures and stragglers
- Better load balancing and resource usage
- Combining of streaming data with static datasets and interactive queries
- Native integration with advanced processing libraries (SQL, machine learning, graph processing)

KAFKA

- In Big Data, an enormous volume of data is used. Regarding data, we have two main challenges.The first challenge is how to collect large volume of data and the second challenge is to analyze the collected data. To overcome those challenges, you must need a messaging system.
- Kafka is high performance,real time messaging system.
- It is an open source tool and part of Apache group

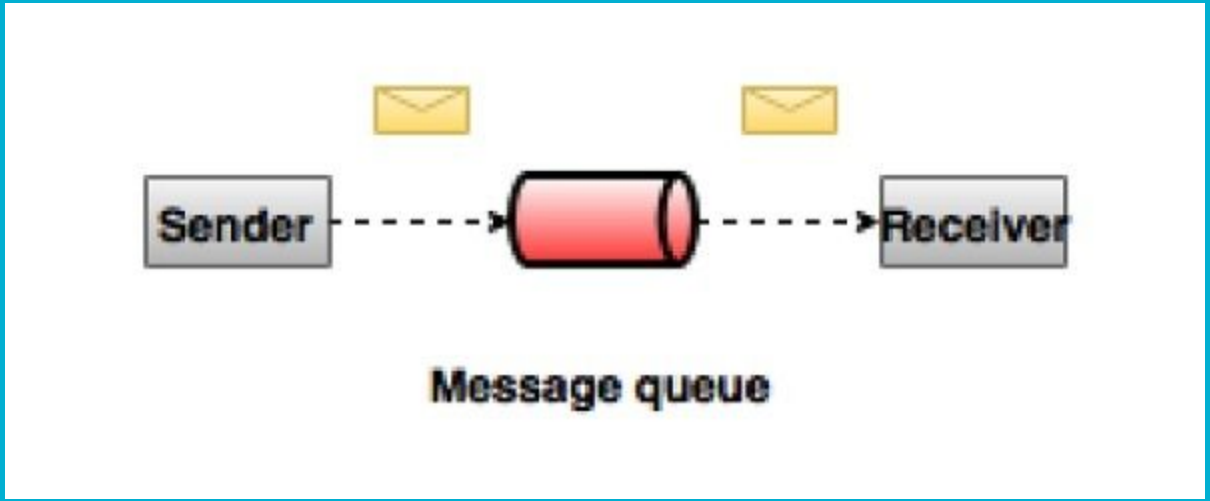
- Kafka is designed for distributed high throughput systems.
- Kafka tends to work very well as a replacement for a more traditional message broker. In comparison to other messaging systems,.
- Kafka has better throughput, built-in partitioning, replication and inherent fault-tolerance, which makes it a good fit for large-scale message processing applications.

What is a Messaging System?

- A Messaging System is responsible for transferring data from one application to another, so the applications can focus on data, but not worry about how to share it.
- Distributed messaging is based on the concept of reliable message queuing. Messages are queued asynchronously between client applications and messaging system.
- Two types of messaging patterns are available − one is point to point and the other is publish-subscribe (pub-sub) messaging system. Most of the messaging patterns follow **pub-sub**.

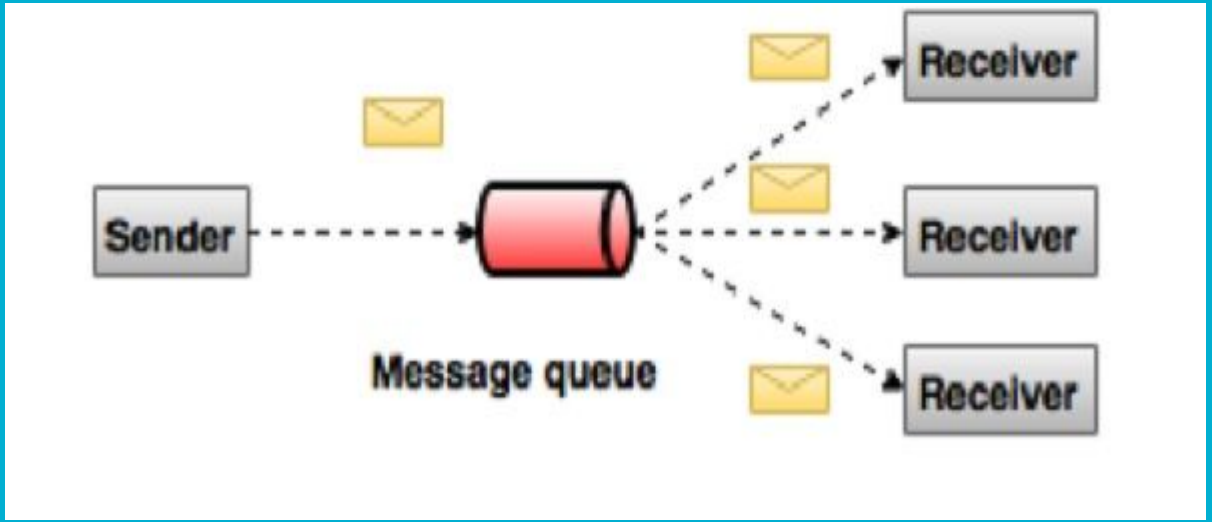
Point to Point Messaging System

- In a point-to-point system, messages are persisted in a queue.
- One or more consumers can consume the messages in the queue, but a particular message can be consumed by a maximum of one consumer only.
- Once a consumer reads a message in the queue, it disappears from that queue. The typical example of this system is an Order Processing System, where each order will be processed by one Order Processor, but Multiple Order Processors can work as well at the same time. .


Sender
Receiver
Message queue

Publish-Subscribe Messaging System

- In the publish-subscribe system, messages are persisted in a topic. Unlike point-to-point system, consumers can subscribe to one or more topic and consume all the messages in that topic.
- In the Publish-Subscribe system, message producers are called publishers and message consumers are called subscribers.
- A real-life example is Dish TV, which publishes different channels like sports, movies, music, etc., and anyone can subscribe to their own set of channels and get them whenever their subscribed channels are available.


Receiver
Sender
Receiver
Message queue
Receiver

What is Kafka?

- Apache Kafka is a distributed publish-subscribe messaging system and a robust queue that can handle a high volume of data and enables you to pass messages from one end-point to another.
- Kafka is suitable for both offline and online message consumption. Kafka messages are persisted on the disk and replicated within the cluster to prevent data loss.
- Kafka is built on top of the ZooKeeper synchronization service. It integrates very well with Apache Storm and Spark for real-time streaming data analysis.

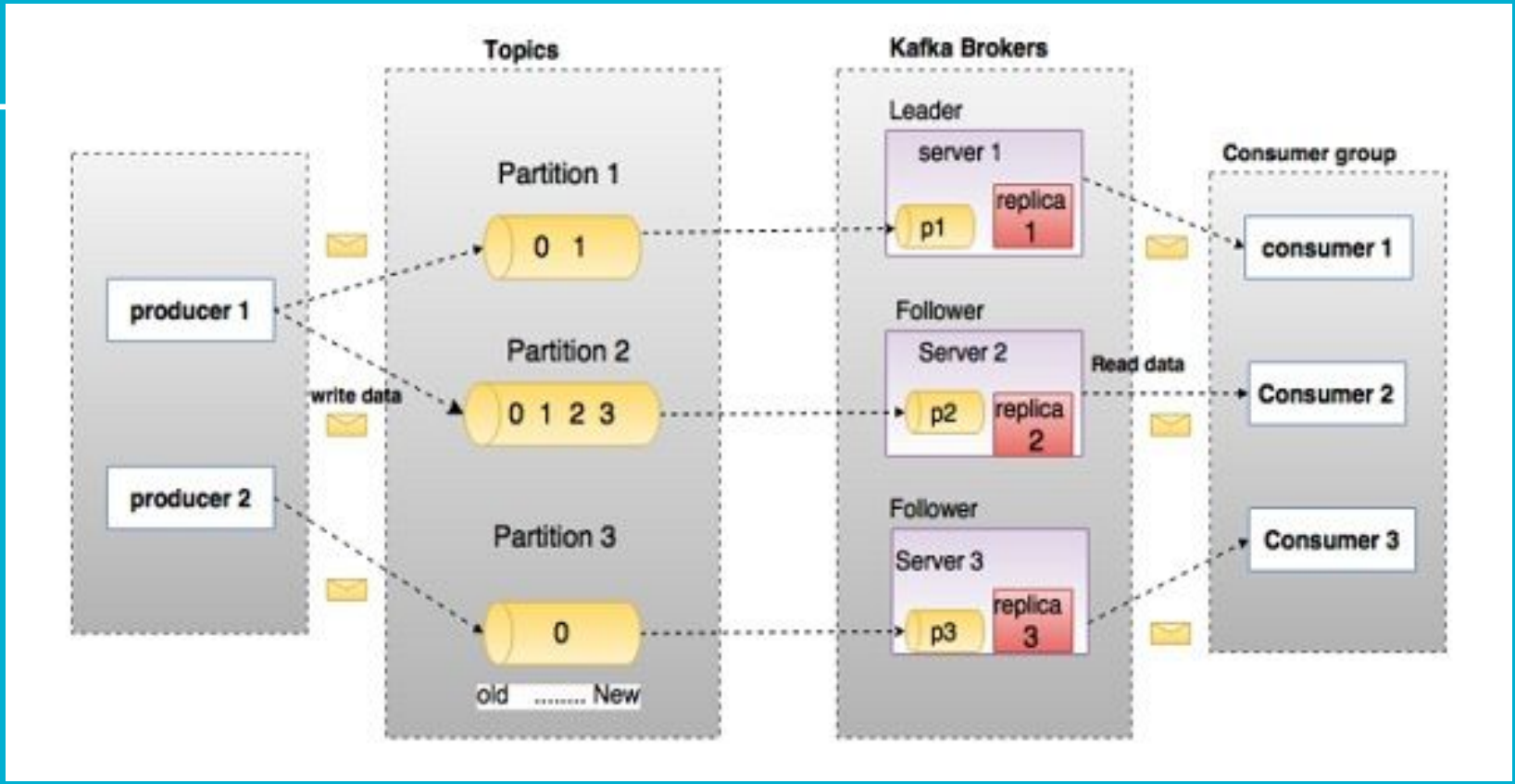
Benefits

Following are a few benefits of Kafka −

- **Reliability** − Kafka is distributed, partitioned, replicated and fault tolerance.
- **Scalability** − Kafka messaging system scales easily without down time..
- **Durability** − Kafka uses Distributed commit log which means messages persists on disk as fast as possible, hence it is durable..
- **Performance** − Kafka has high throughput for both publishing and subscribing messages. It maintains stable performance even many TB of messages are stored.

Need for Kafka

- Kafka is a unified platform for handling all the real-time data feeds.
- Kafka supports low latency message delivery and gives guarantee for fault tolerance in the presence of machine failures.
- It has the ability to handle a large number of diverse consumers. Kafka is very fast, performs 2 million writes/sec.
- Kafka persists all data to the disk, which essentially means that all the writes go to the page cache of the OS (RAM). This makes it very efficient to transfer data from page cache to a network socket.


Topics
Kafka Brokers
Leader
server 1
Partition 1
0 1
p1
replica 1
Consumer group
consumer 1
producer 1
Follower
Server 2
Partition 2
Read data
write data
0 1 2 3
p2
replica 2
Consumer 2
producer 2
Follower
Partition 3
Server 3
Consumer 3
0
p3
replica 3
old ..... New

Topics

A stream of messages belonging to a particular category is called a topic. Data is stored in topics.Topics are split into partitions. For each topic, Kafka keeps a mini-mum of one partition. Each such partition contains messages in an immutable ordered sequence. A partition is implemented as a set of segment files of equal sizes.

Partition

Topics may have many partitions, so it can handle an arbitrary amount of data.

Partition offset

Each partitioned message has a unique sequence id called as offset.

Replicas of partition

Replicas are nothing but backups of a partition. Replicas are never read or write data. They are used to prevent data loss.

Brokers

- Brokers are simple system responsible for maintaining the pub-lished data. Each broker may have zero or more partitions per topic. Assume, if there are N partitions in a topic and N number of brokers, each broker will have one partition.
- Assume if there are N partitions in a topic and more than N brokers (n + m), the first N broker will have one partition and the next M broker will not have any partition for that particular topic.
- Assume if there are N partitions in a topic and less than N brokers (n-m), each broker will have one or more partition sharing among them. This scenario is not recommended due to unequal load distri-bution among the broker.

Kafka Cluster

Kafka's having more than one broker are called as Kafka cluster. A Kafka cluster can be expanded without downtime. These clusters are used to manage the persistence and replication of message data.

Producers

Producers are the publisher of messages to one or more Kafka topics. Producers send data to Kafka brokers. Every time a producer pub-lishes a message to a broker, the broker simply appends the message to the last segment file. Actually, the message will be appended to a partition. Producer can also send messages to a partition of their choice.

Consumers

Consumers read data from brokers. Consumers subscribes to one or more topics and consume published messages by pulling data from the brokers.

Leader

Leader is the node responsible for all reads and writes for the given partition. Every partition has one server acting as a leader.

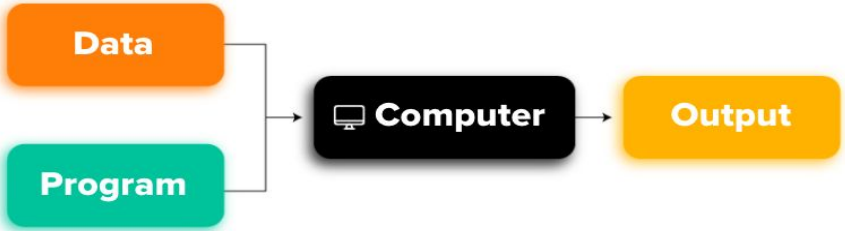
Follower

Node which follows leader instructions are called as follower. If the leader fails, one of the follower will automatically become the new leader. A follower acts as normal consumer, pulls messages and up-dates its own data store.

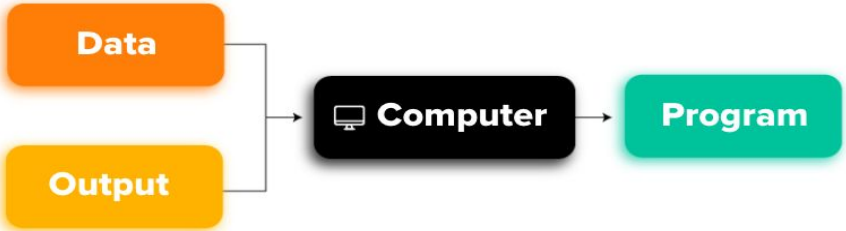
Streaming Ecosystem

Solution To a problem

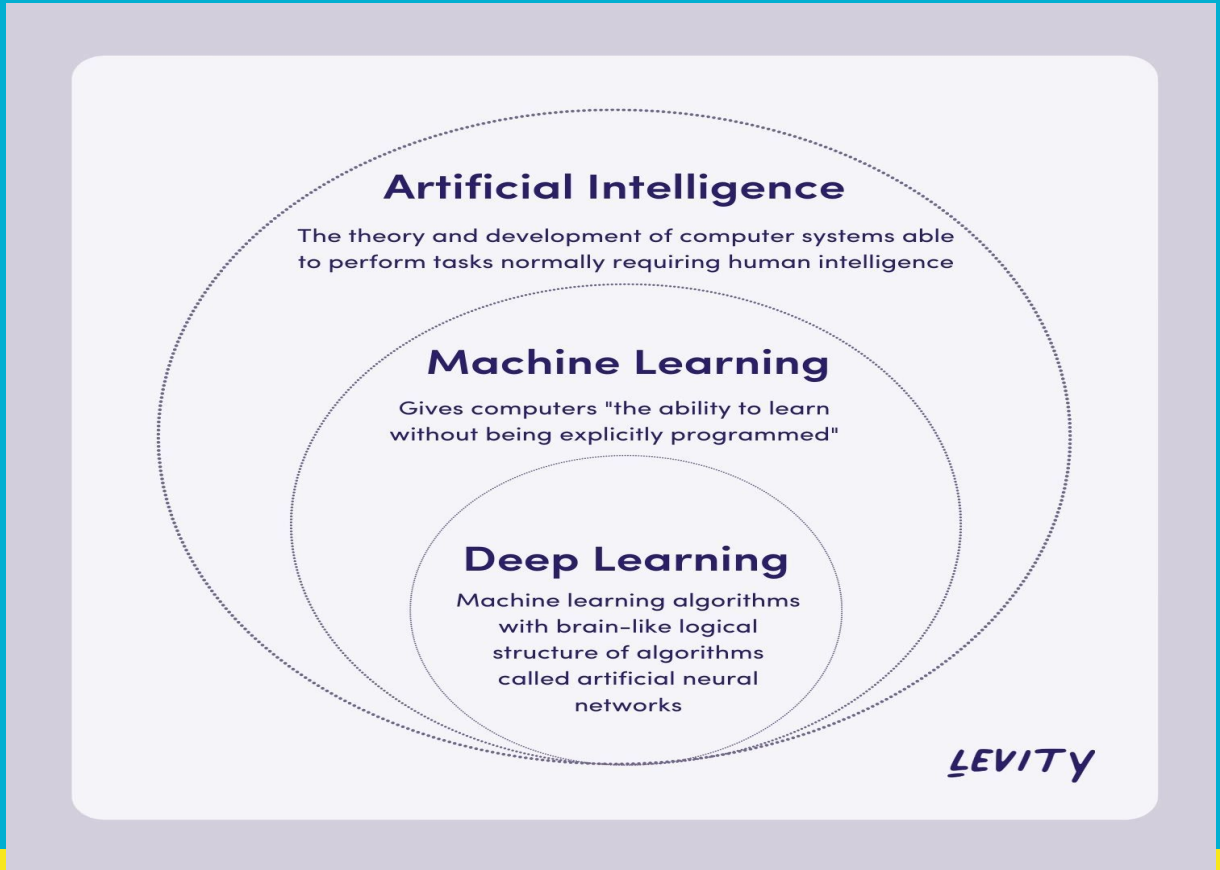
TRADITIONAL PROGRAMMING



MACHINE LEARNING



Machine Learning



- Learning : Ability to improve one’s behaviour with experience.
- Machine learning explores algorithms that learn from data, build model from data and this model can be used for solving task or Decision making.

Mahout

- **Hadoop** is an open-source framework from Apache that allows to store and process big data in a distributed environment across clusters of computers using simple programming models.
- Apache **Mahout** is an open source project that is primarily used for creating scalable machine learning algorithms. It implements popular machine learning techniques such as:
- Recommendation
- Classification
- Clustering
- Apache Mahout started as a sub-project of Apache's Lucene in 2008. In 2010, Mahout became a top level project of Apache.

Features of Mahout

The primitive features of Apache Mahout are listed below.

- The algorithms of Mahout are written on top of Hadoop, so it works well in distributed environment. Mahout uses the Apache Hadoop library to scale effectively in the cloud.
- Mahout offers the coder a ready-to-use framework for doing data mining tasks on large volumes of data.
- Mahout lets applications to analyze large sets of data effectively and in quick time.
- Includes several MapReduce enabled clustering implementations such as k-means, fuzzy k-means, Canopy, Dirichlet, and Mean-Shift.
- Supports Distributed Naive Bayes and Complementary Naive Bayes classification implementations.
- Comes with distributed fitness function capabilities for evolutionary programming.
- Includes matrix and vector libraries.

Applications of Mahout

- Companies such as Adobe, Facebook, LinkedIn, Foursquare, Twitter, and Yahoo use Mahout internally.
- Foursquare helps you in finding out places, food, and entertainment available in a particular area. It uses the recommender engine of Mahout.
- Twitter uses Mahout for user interest modelling.
- Yahoo! uses Mahout for pattern mining.

Machine Learning with Spark

https://spark.apache.org/mllib/

https://www.databricks.com/spark/getting-started-with-apache-spark/machine-learning

Naive Bayes: